人工智能导论
第七章 强化学习
Reinforcement Learning

西安交通大学
陈炜煌 助理教授

人工智能学院

人工智能基础方法与应用

AlphaGo：强化学习代表性成果1

- 由 DeepMind开发的围棋人工智能系统
- 2016 年 3 月，AlphaGo 在五番棋比赛中以 4:1 战胜韩国顶尖职业棋手李世石九段，成为人工智能发展史上的标志性事件
- 2017 年，在中国乌镇围棋峰会上，AlphaGo 以 3:0 战胜当时世界排名第一的中国棋手柯洁

AlphaGo

4:1

2016.03



李世石

DeepSeekR1：强化学习代表性成果2

- DeepSeek系列大模型是我国DeepSeek公司推出的通用大语言模型，主要用于文本生成、文本理解、对话交互等任务
- DeepSeek-R1 在后训练阶段大规模使用了强化学习技术，在仅有极少标注数据的情况下，极大提升了模型推理能力。在数学、代码、自然语言推理等任务上，性能比肩 OpenAI o1 正式版



人工智能基础方法与应用

AI人机：强化学习代表性成果3

- 2019年12月20号，腾讯 AI Lab 发布《Mastering Complex Control in MOBA Games with Deep Reinforcement Learning》，利用深度强化学习技术训练出超强AI“绝悟”，能够击败顶尖职业人类玩家

Hero	DiaoChan	DiRenjie	LuNa	HanXin	HuaMulan
Hero Type	Mage	Marksman	Warrior+Mage	Assassin	Warrior
Score (BO5)	3:0 (AI:Professional)	3:0 (AI:Professional)	3:0 (AI:Professional)	3:1 (AI:Professional)	3:0 (AI:Professional)
Kill	5.0:1.3	2.3:0.7	2.7:1.0	2.5:1.5	4.0:1.3
Game Length	6’56”	6’23”	7’53”	6’41”	6’48”
Gold/min	852.7:430.6	869.3:606.6	969.7:724.0	954.1:754.2	945.2:654.2
Exp/min	900.0:573.0	895.3:661.7	979.0:817.2	965.4:802.5	921.4:723.1

AI与职业游戏选手对战结果

人工智能基础方法与应用

强化学习很难?

- 强化学习到底在解决什么问题?
- 和我们之前学习的监督学习有什么不同?
- 为什么可以放在机器学习的统一框架中理解?

强化学习的核心思想很简单!
在微电子的学习中可以用到!

人工智能基础方法与应用

目录

RL是什么？（ML的三步骤）
策略梯度
演员-评论员
设计奖励
无奖励：从演示中学习

人工智能基础方法与应用

强化学习：在互动中学习

机器学习 ≈ 寻找从输入到输出的映射函数



智能体

动作 = f(观测)

观测

映射函数
输入

动作

映射函数
输出

寻找**策略函数f**

以最大化长期累积奖励

奖励

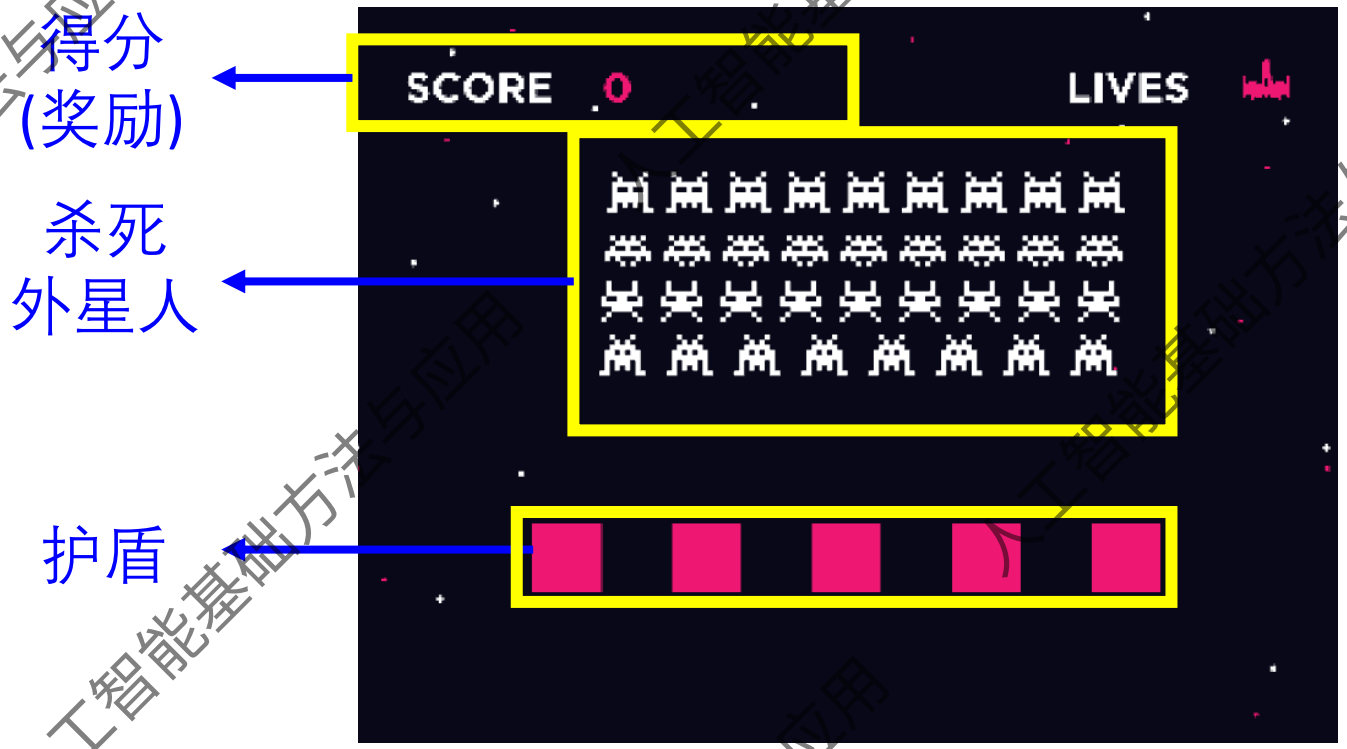


环境

人工智能基础方法与应用

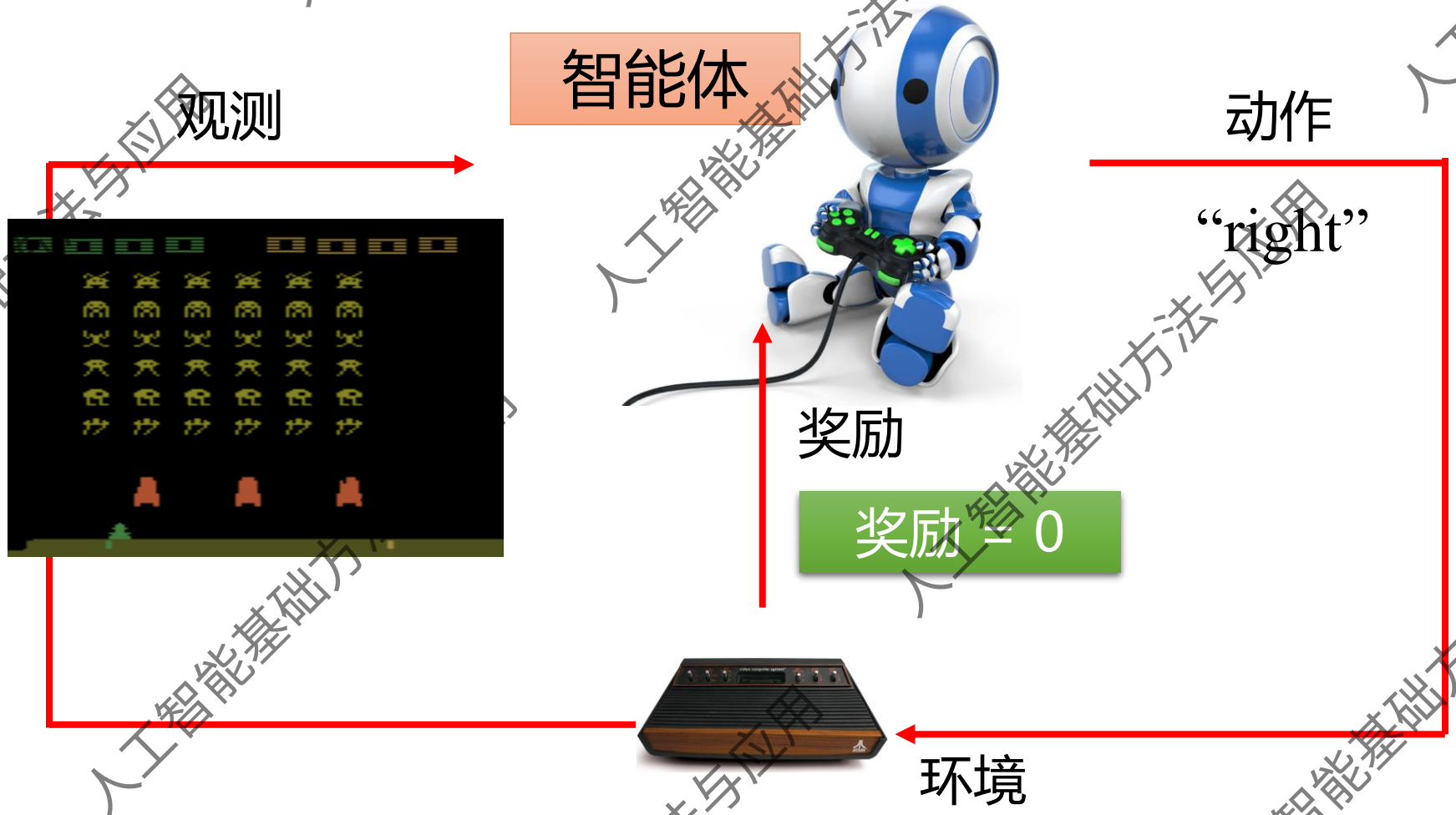
例子1: 用Space Invader理解RL

终止条件：所有外星人被消灭，或者宇宙飞船被摧毁。



动作集： ← fire →

例子1: 用Space Invader理解RL



人工智能基础方法与应用

例子1: 用Space Invader理解RL

寻找策略函数最大化期望奖励。



观测



智能体

动作

“fire”

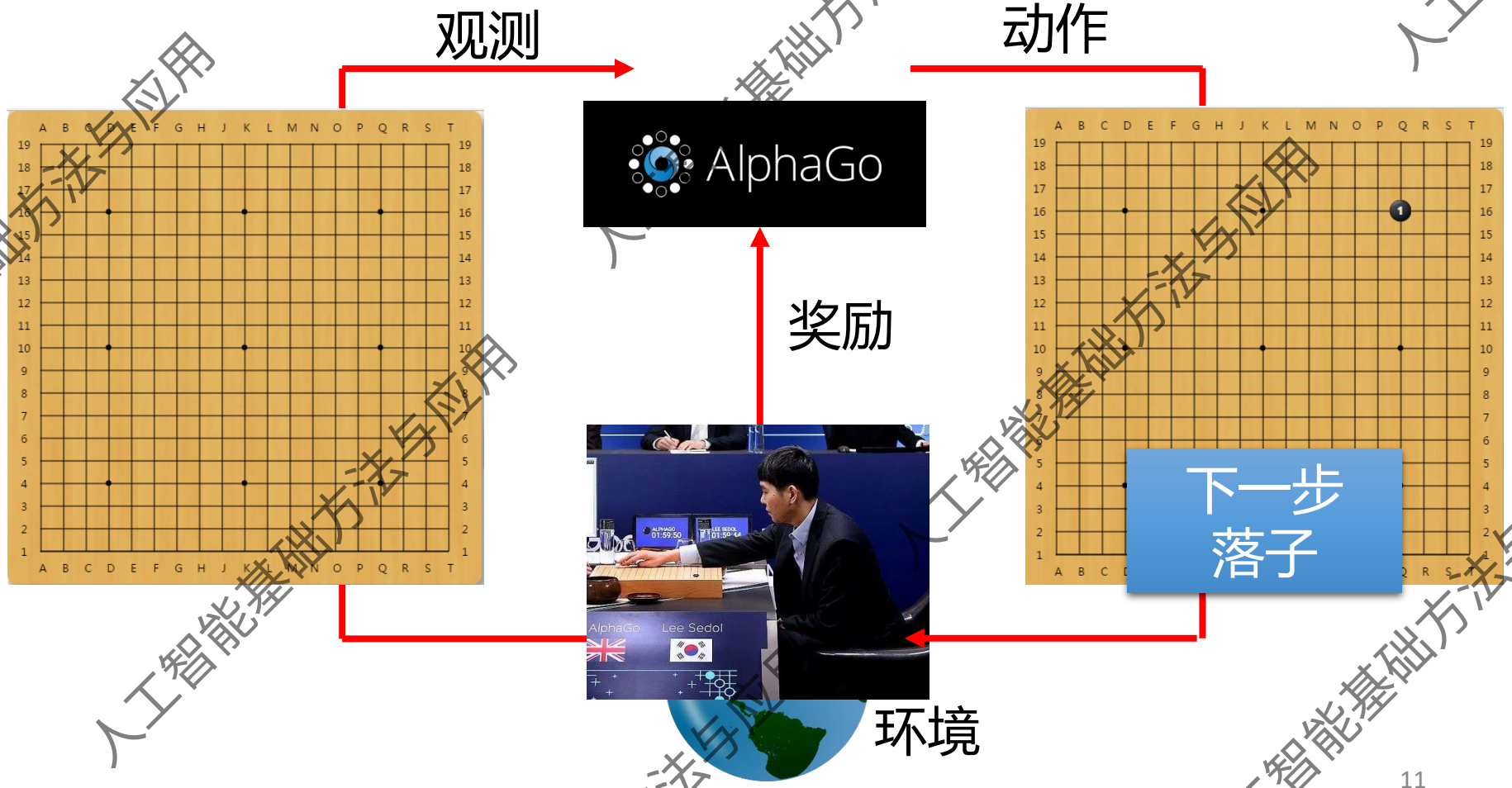
奖励

奖励 = 5
杀死外星人



环境

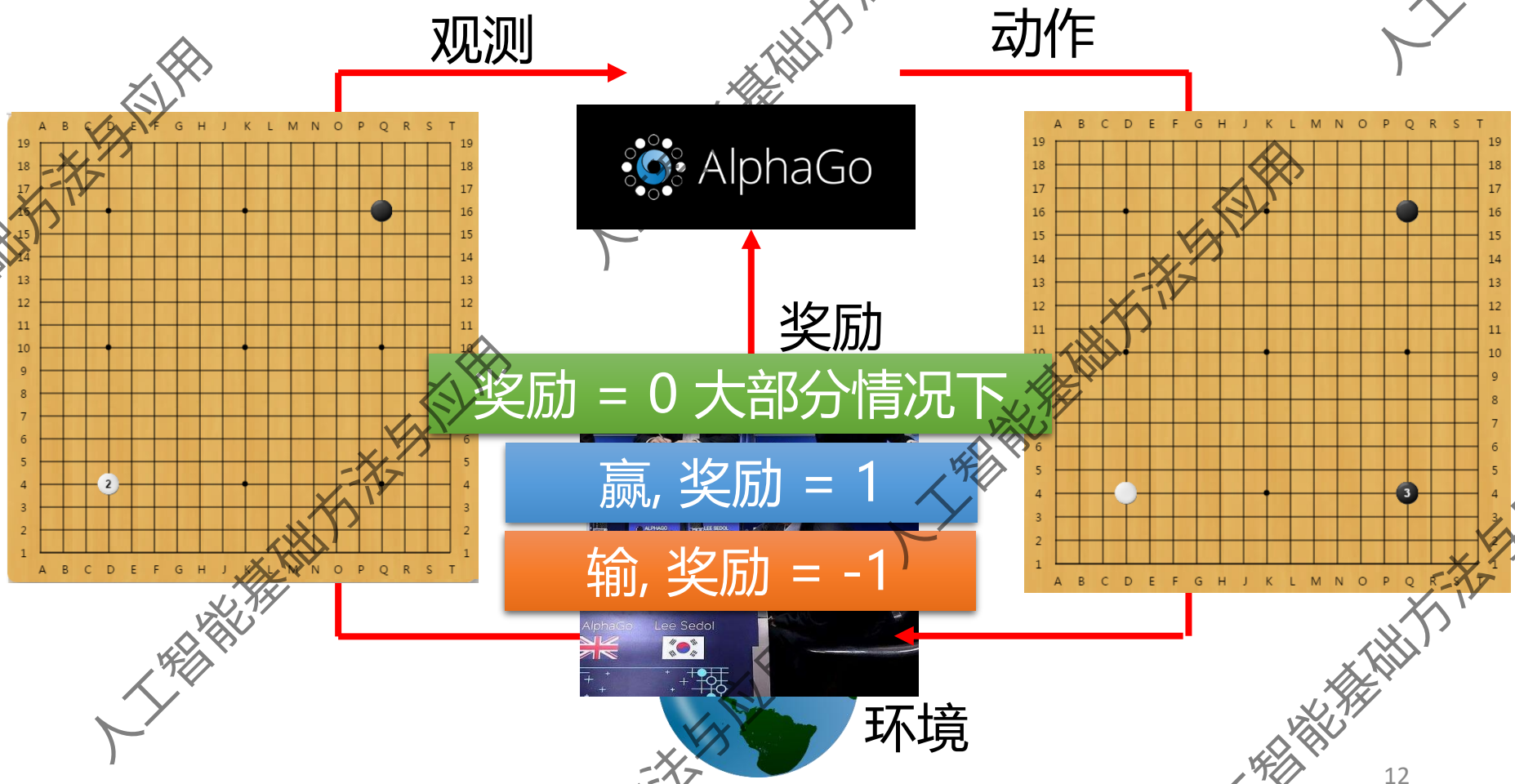
例子2: 下围棋



人工智能基础方法与应用

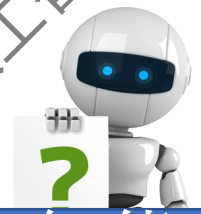
例子2: 下围棋

寻找策略函数最大化期望奖励。



人工智能基础方法与应用

黑箱，无法控制

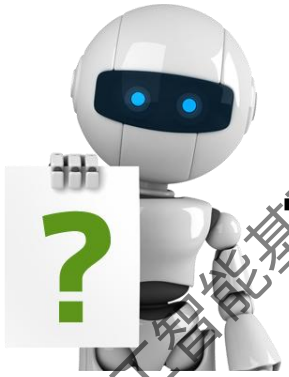


	智能体	环境	奖励函数
视频游戏			杀死外星人 获得5分
围棋	AlphaGo Google DeepMind		围棋 规则

人工智能基础方法与应用

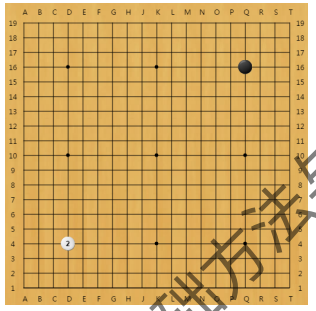
监督学习 *VS* 强化学习





Human label

? ⟷ Cat





Human label

? ⟷ “3-3”?

监督学习：针对“标准答案”已知的情况

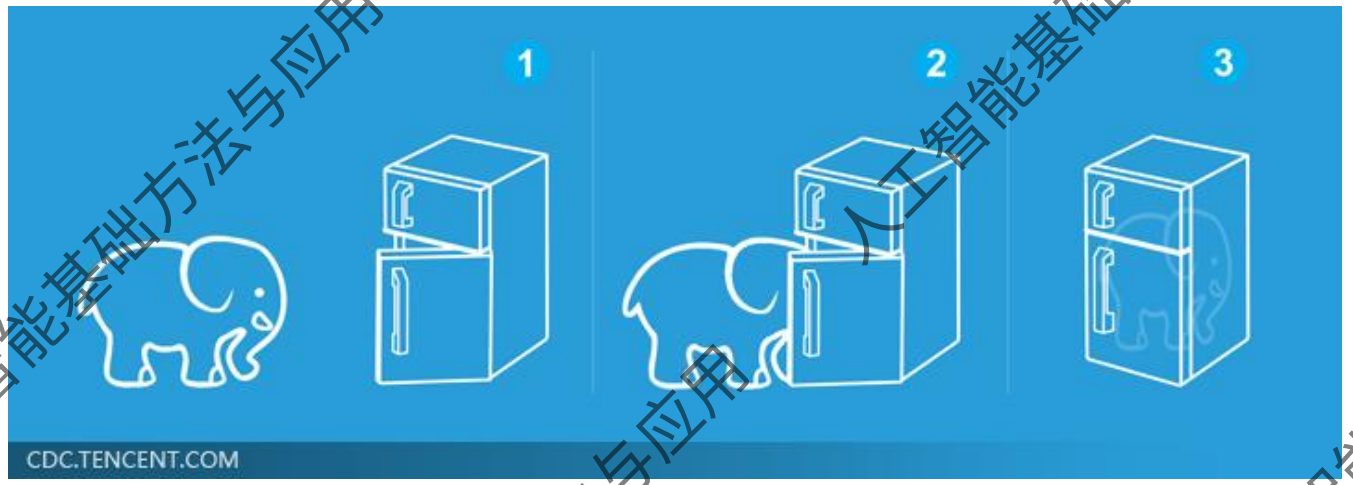
强化学习：针对“标准答案”**未知**的情况

机器学习三步骤

Step 1
定义模型函数

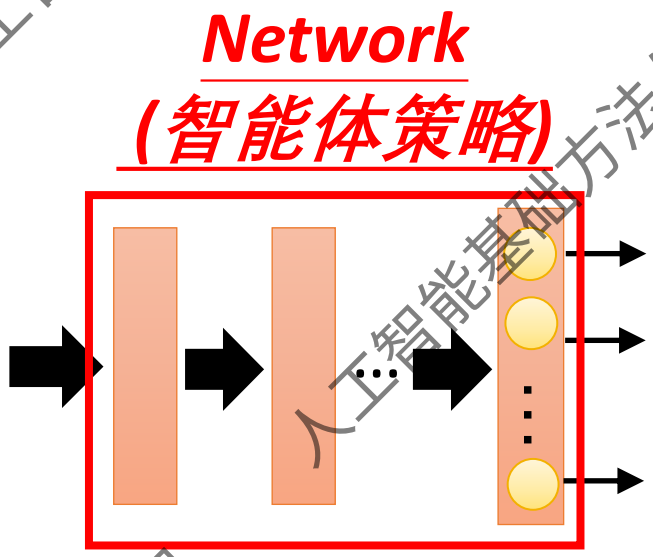
Step 2
定义损失函数

Step 3
优化模型参数



人工智能基础方法与应用

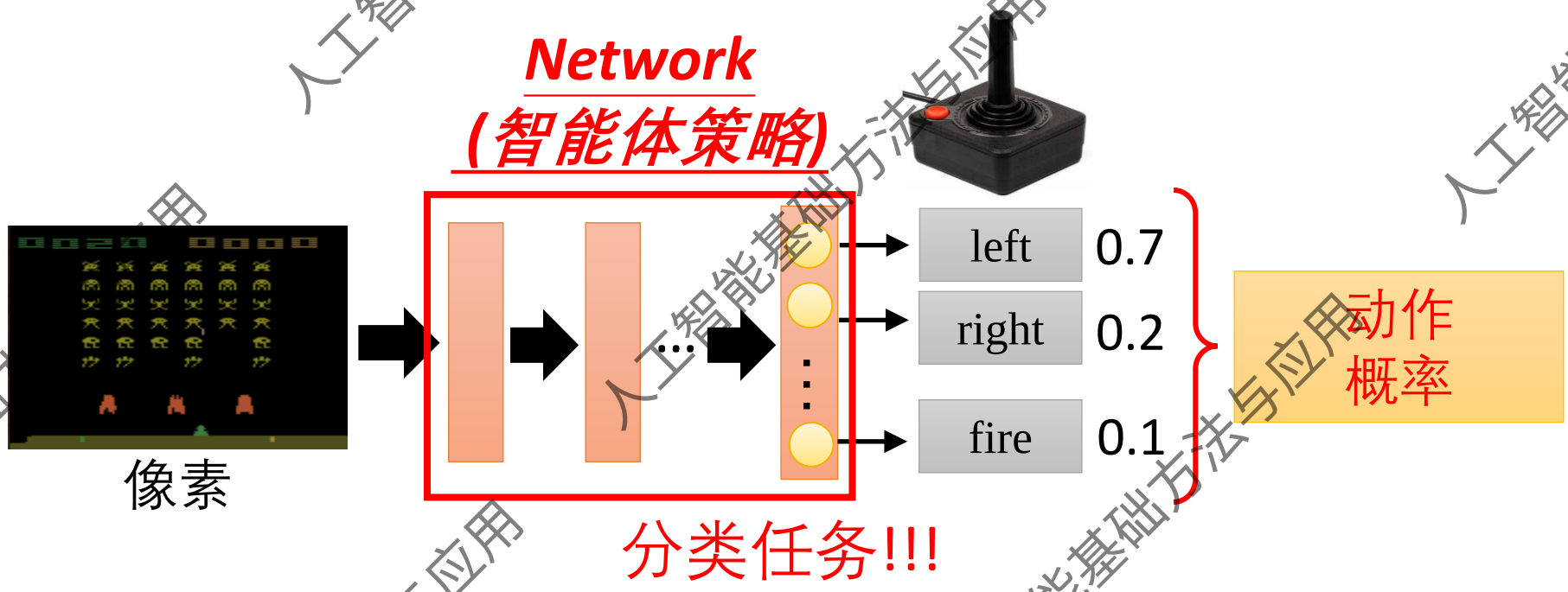
Step 1:定义模型函数



- 智能体策略，π_θ
  - 确定性策略：智能体直接采取最有可能的动作，查找表
  - 随机性策略：CNN（图像）、RNN（历史画面）、Transformer（上下文）

人工智能基础方法与应用

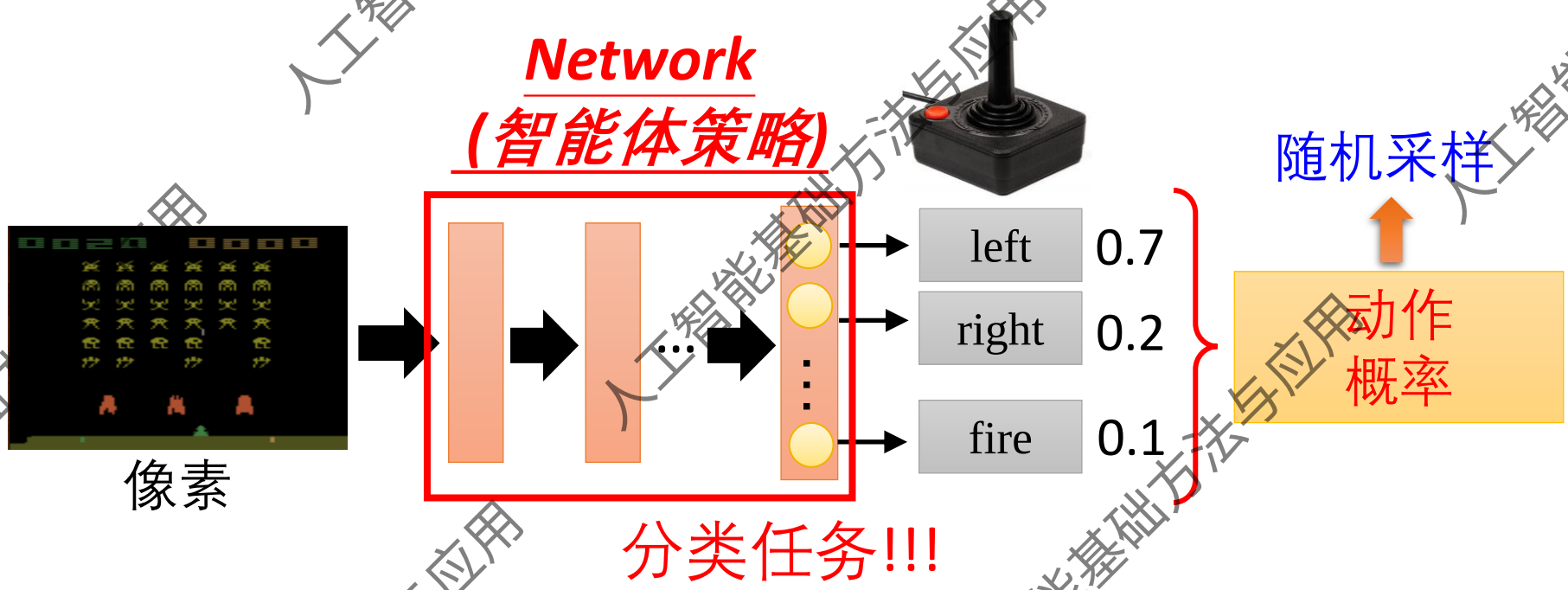
Step 1:定义模型函数



- 输入：智能体从环境中获得的观测信息，通常表示为向量或矩阵
- 输出：可选动作，以及该动作被选择的概率

人工智能基础方法与应用

Step 1:定义模型函数



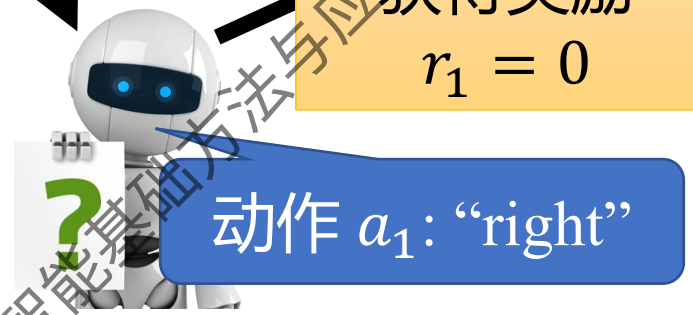
- 基于动作概率的随机采样？只做当前看起来最好的动作，很快会陷入局部最优 → 强化学习不仅要利用当前看起来好的动作，还要通过随机探索发现潜在更好的动作

Step 2: 定义损失函数

开始：观测 s_1



获得奖励
$r_1 = 0$

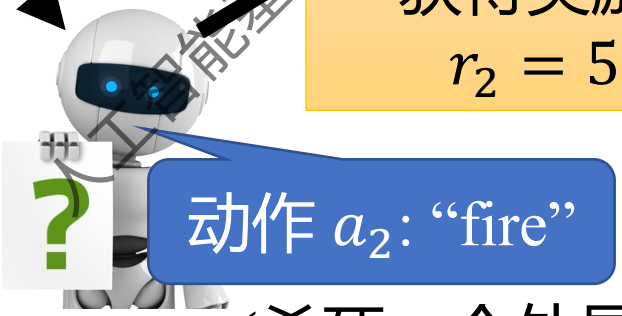


动作 a_1: “right”

观测 s_2



获得奖励
$r_2 = 5$



动作 a_2: “fire”

(杀死一个外星人)

观测 s_3



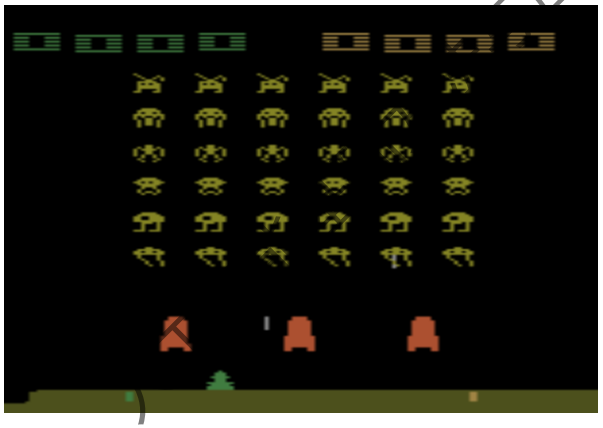
人工智能基础方法与应用

Step 2: 定义损失函数

开始：观测 s_1



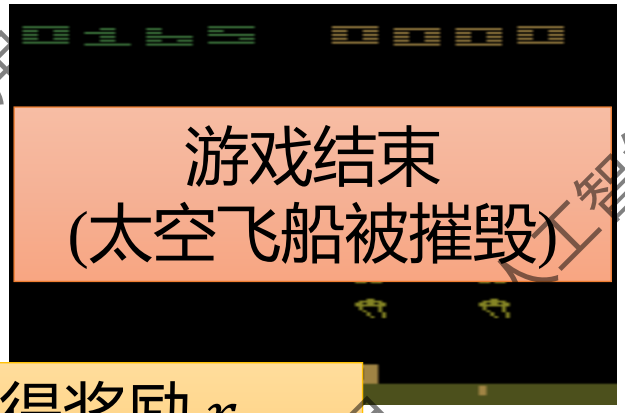
观测 s_2



观测 s_3



多轮互动后



获得奖励 r_T

动作 a_T

形成一回合***episode***

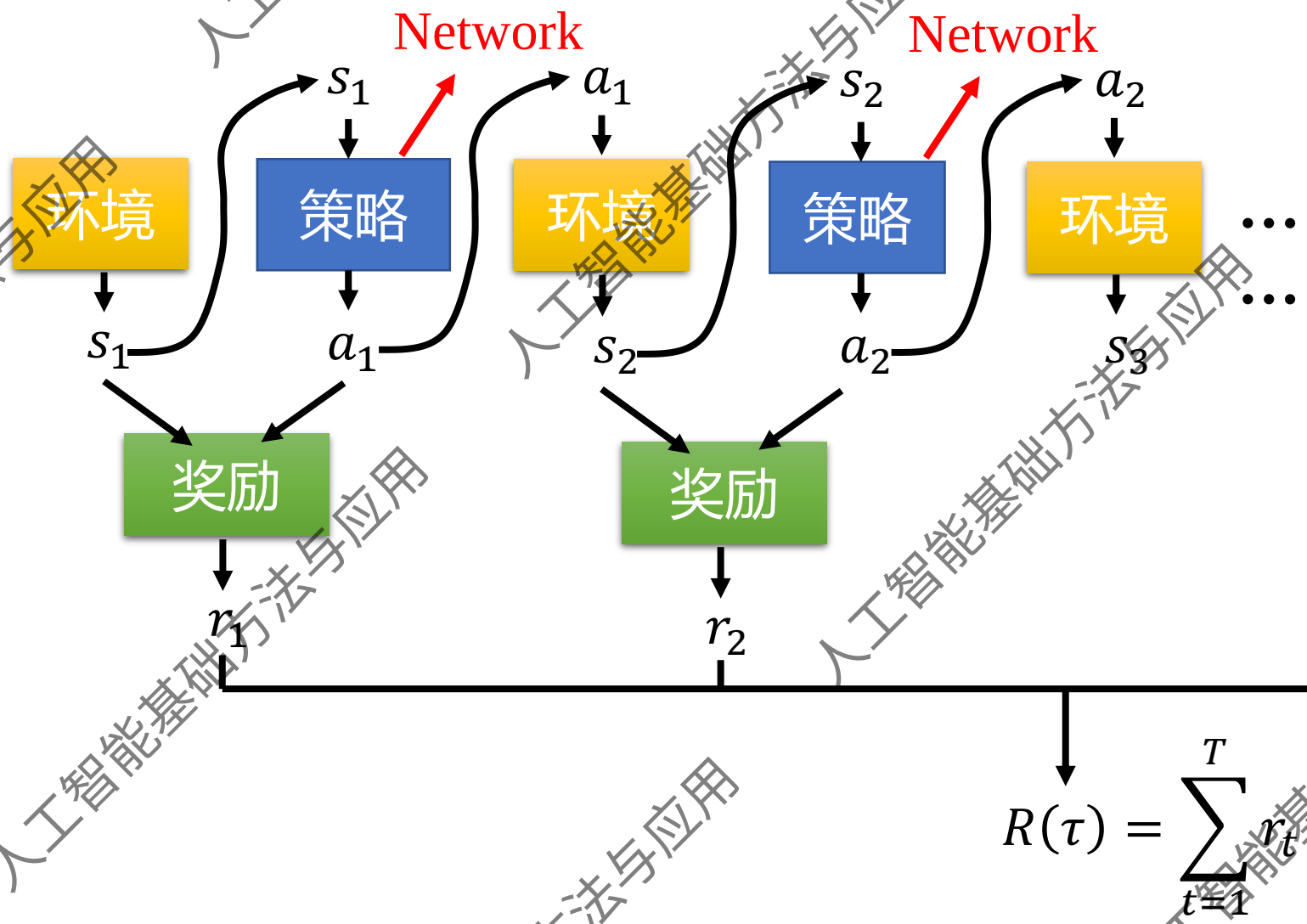
得到累积奖励

return:

$$R = \sum_{t=1}^{T} r_t$$

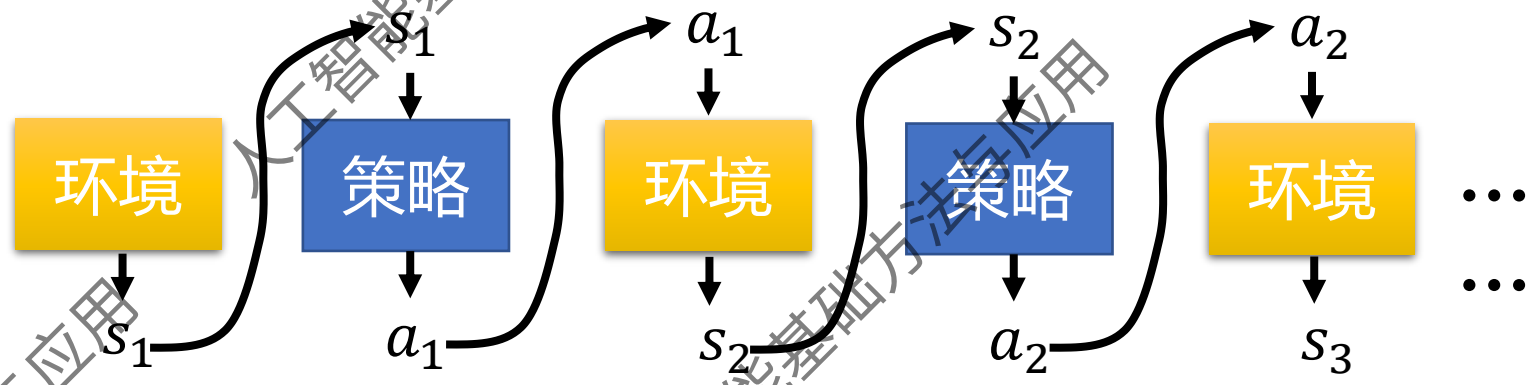
（***Loss = -R***）最大化的目标

Step 3: 优化模型参数



人工智能基础方法与应用

Step 3: 优化模型参数



轨迹 $\tau = \{s_1, a_1, s_2, a_2, \cdots, s_T, a_T\}$

给定智能体策略参数θ，轨迹τ的概率分布是：

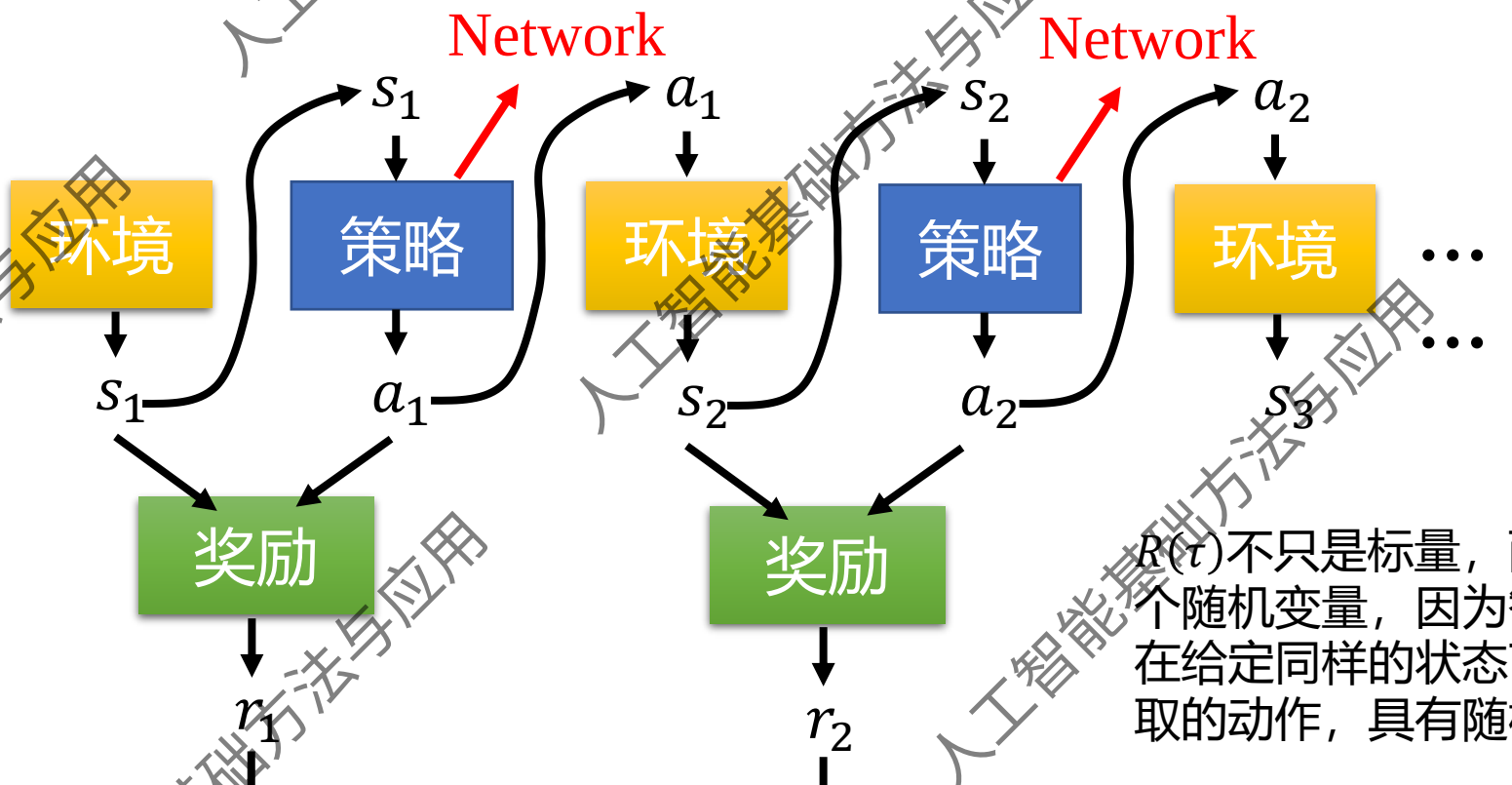
$$p_\theta(\tau)$$

$$= p(s_1)p_\theta(a_1|s_1)p(s_2|s_1,a_1)p_\theta(a_2|s_2)p(s_3|s_2,a_2)\cdots$$

$$= p(s_1)\prod_{t=1}^{T} p_\theta(a_t|s_t)p(s_{t+1}|s_t,a_t)$$

人工智能基础方法与应用

Step 3: 优化模型参数



$R(\tau)$不只是标量，而是一个随机变量，因为智能体在给定同样的状态下会采取的动作，具有随机性

期望奖励

$$\bar{R}_\theta = \sum_\tau R(\tau) p_\theta(\tau) = E_{\tau \sim p_\theta(\tau)}[R(\tau)]$$

$$R(\tau) = \sum_{t=1}^{T} r_t$$

Step 3: 优化模型参数

- 为什么强化学习的优化困难
  - 智能体动作不是固定输出，而是从策略分布中采样得到
  - 环境的反馈也不完全确定，相同动作可能带来不同结果
  - 环境通常无法直接建模和求导，只能通过交互获得反馈

强化学习的核心难点
如何在随机、黑箱、交互式环境中完成策略优化

人工智能基础方法与应用

强化学习的基本概念1

序号	概念	定义
1	智能体（Agent）	强化学习中的学习主体
2	环境（Environment）	智能体所处的外部世界
3	观测（Observation）	从环境中获得的信息，用于描述当前状态
4	动作（Action）	根据当前观测做出的操作
5	回合（Episode）	从初始观测直到任务结束的完整交互过程
6	轨迹（Trajectory）	一个回合中，产生的状态、动作序列
7	策略（Policy）	行为规则，决定在某个观测下应该采取什么动作
8	奖励（Reward）	执行一次动作后，环境给出的立即标量反馈
9	累积奖励（Return）	一个回合中，所有即时奖励的总和

人工智能基础方法与应用

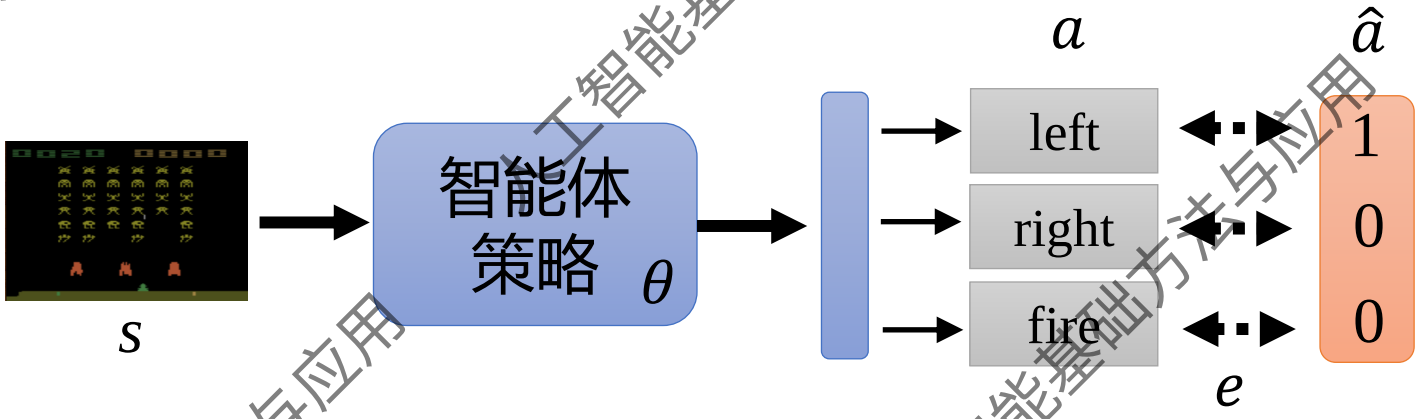
目录

- RL是什么？（ML的三步骤）
- 策略梯度
- 演员-评论员
- 设计奖励
- 无奖励：从演示中学习

人工智能基础方法与应用

如何控制智能体的动作

- 给定特定的观测 s，使得采取特定动作$\hat{a}$的概率最大



希望采取动作$\hat{a}$时

$$L = e$$

不希望采取动作$\hat{a}$时

$$L = -e$$

$$\theta^* = arg\min_{\theta} L$$

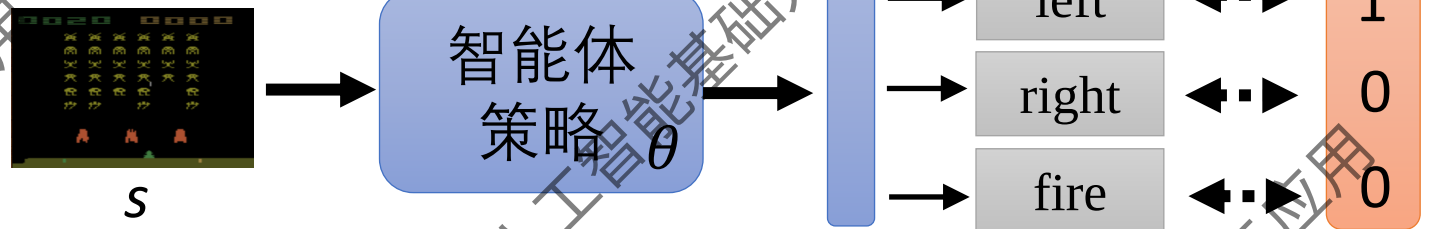
如何控制智能体的动作

- 容易混淆的问题：不希望做某个动作 ≠ 什么都不做
  - 当我们说“不希望智能体采取某个动作”时，并不是让智能体停止行动，而是希望它转向其他可选动作
  - 如果希望“什么都不做”也成为一种选择，那么“保持不动”本身也必须被定义为一个动作
  - 例如，在 Space Invaders 游戏中，若动作集合只有向左、向右、开火三个动作，智能体每一步就只能从这三个动作中选择。若希望它还可以原地不动，就需要将原地不动加入动作集合

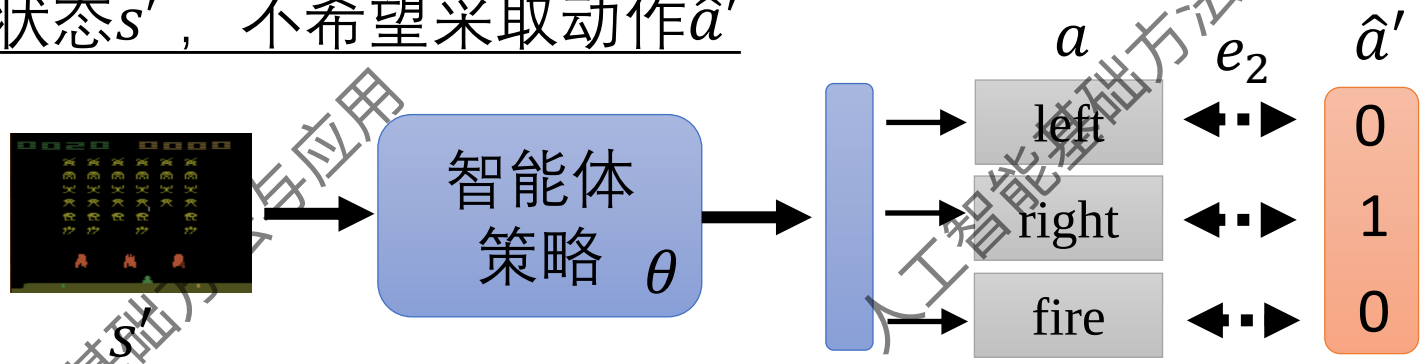
人工智能基础方法与应用

如何控制智能体的动作

给定状态s，希望采取动作 $\hat{a}$



给定状态s'，不希望采取动作$\hat{a}'$



$$L = e_1 - e_2 \qquad \theta^* = arg\min_{\theta} L$$

人工智能基础方法与应用

如何控制智能体的动作

训练数据

$\{s_1, \hat{a}_1\}$ +1 Yes

$\{s_2, \hat{a}_2\}$ -1 No

$\{s_3, \hat{a}_3\}$ +1 Yes

⋮ ⋮

$\{s_N, \hat{a}_N\}$ -1 No



$$L = +e_1 - e_2 + e_3 \cdots - e_N$$

$$\theta^* = arg\min_{\theta} L$$

- 让智能体与环境互动，收集轨迹样本和对应奖励
- 根据奖励判断哪些动作应该被鼓励，哪些动作应该被抑制
- 把奖励转化为一个带权重的监督学习问题来更新智能体

如何控制智能体的动作

训练数据

$\{s_1, \hat{a}_1\}$	A_1	+1.5
$\{s_2, \hat{a}_2\}$	A_2	- 0.5
$\{s_3, \hat{a}_3\}$	A_3	+0.5
⋮ ⋮	⋮	
$\{s_N, \hat{a}_N\}$	A_N	-10

? ?

训练数据从哪里来　　权重如何确定

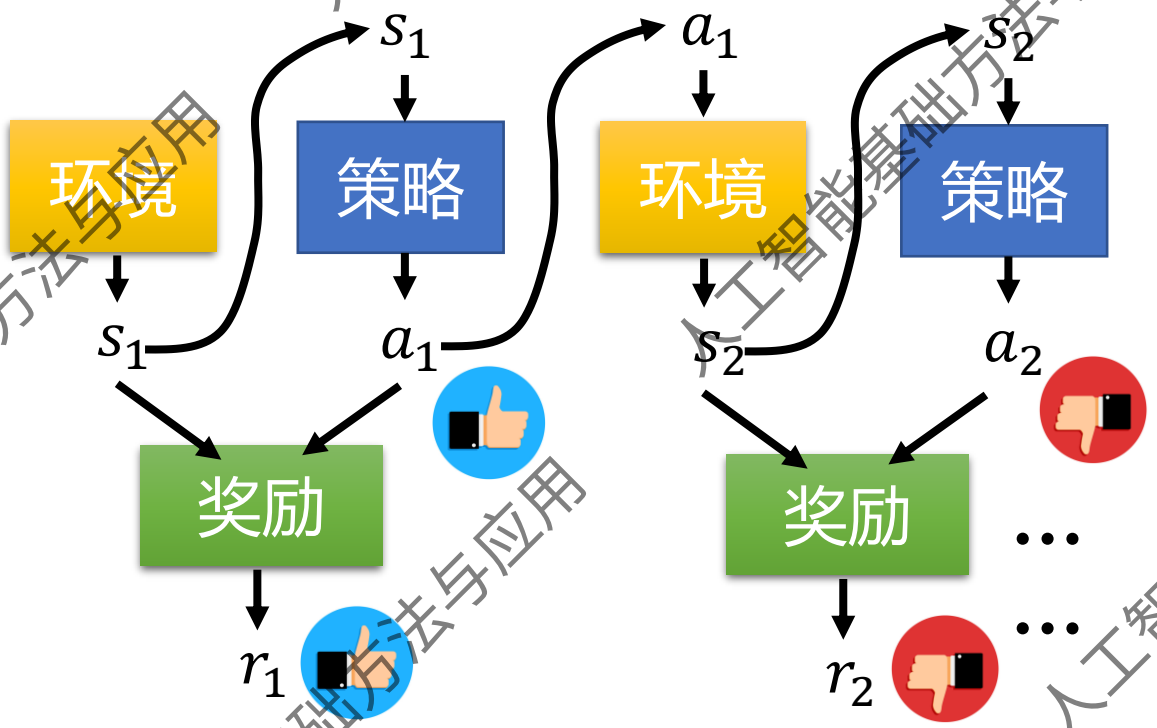


$$L = \sum A_n e_n$$

$$\theta^* = arg\min_{\theta} L$$

人工智能基础方法与应用

智能体0-只看短期奖励



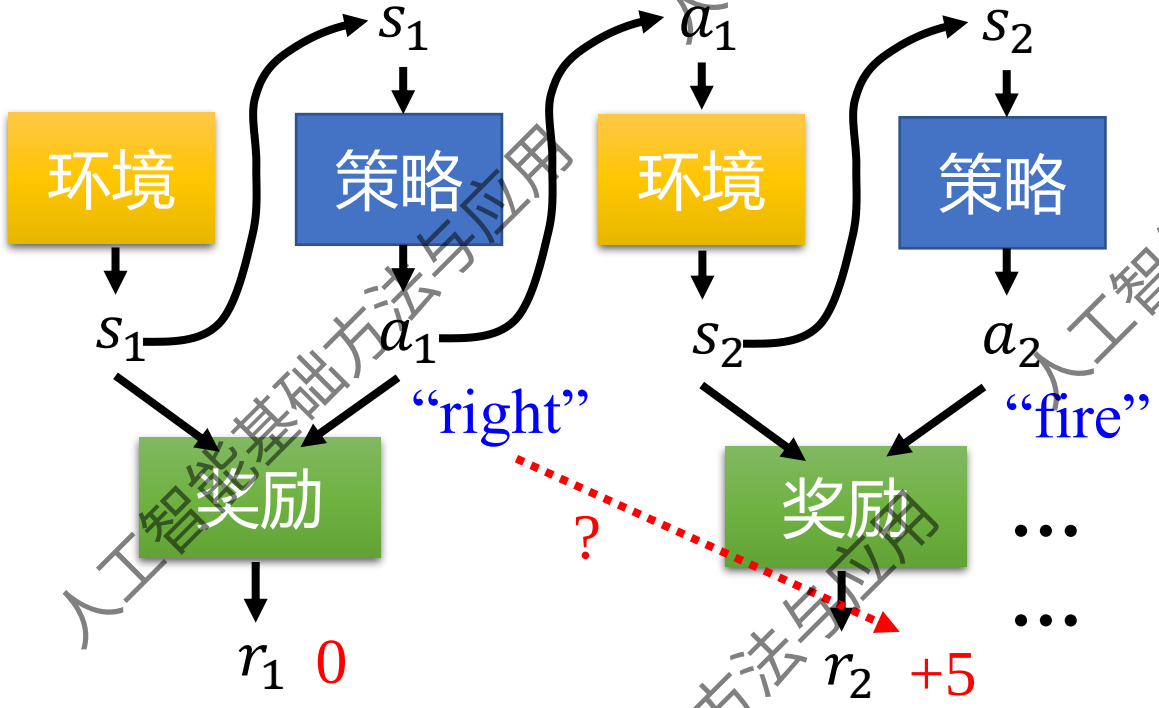
训练数据

$\{s_1, a_1\}$	$A_1 = r_1$
$\{s_2, a_2\}$	$A_2 = r_2$
$\{s_3, a_3\}$	$A_3 = r_3$
⋮	⋮
$\{s_N, a_N\}$	$A_N = r_N$

多回合

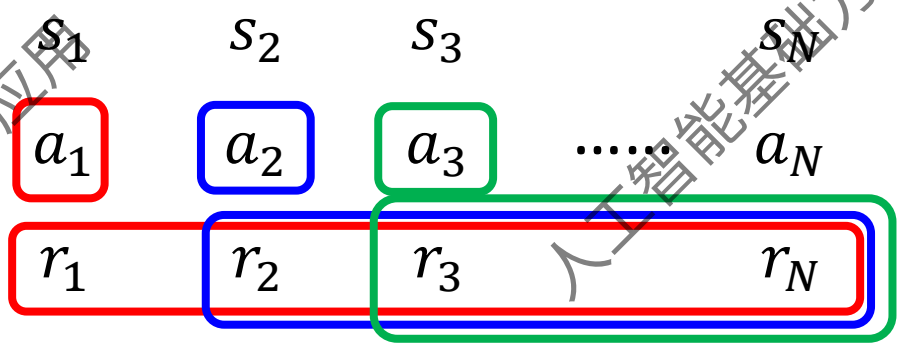
急功近利版本!

智能体0



- 一个动作会影响后续的观测，进一步影响后续奖励
- *奖励延迟*：智能体有时必须牺牲眼前的即时奖励，才能获得更长期的高奖励
- 在《太空侵略者》游戏中，只有“开火”这个动作会带来正奖励，因此智能体0会学出一个总是“开火”的策略

智能体1-处理奖励延迟



$$G_1 = r_1 + r_2 + r_3 + \ldots\ldots + r_N$$

$$G_2 = r_2 + r_3 + \ldots\ldots + r_N$$

$$G_3 = r_3 + \ldots\ldots + r_N$$

累积奖励

训练数据

$$\{s_1, a_1\} \quad A_1 = G_1$$

$$\{s_2, a_2\} \quad A_2 = G_2$$

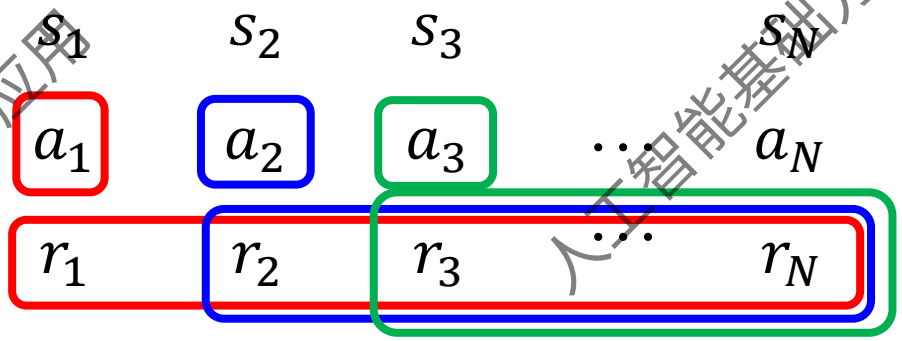
$$\{s_3, a_3\} \quad A_3 = G_3$$

$$\vdots \qquad \vdots$$

$$\{s_N, a_N\} \quad A_N = G_N$$

$$G_t = \sum_{n=t}^{N} r_n$$

智能体2-平衡短期和长期奖励



与归功于a_1?

$$G_1 = r_1 + r_2 + r_3 + \ldots\ldots + r_N$$

$$G_1' = r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots\ldots$$

折扣因子$\gamma < 1$

训练数据

$\{s_1, a_1\}$ $A_1 = G_1'$

$\{s_2, a_2\}$ $A_2 = G_2'$

$\{s_3, a_3\}$ $A_3 = G_3'$

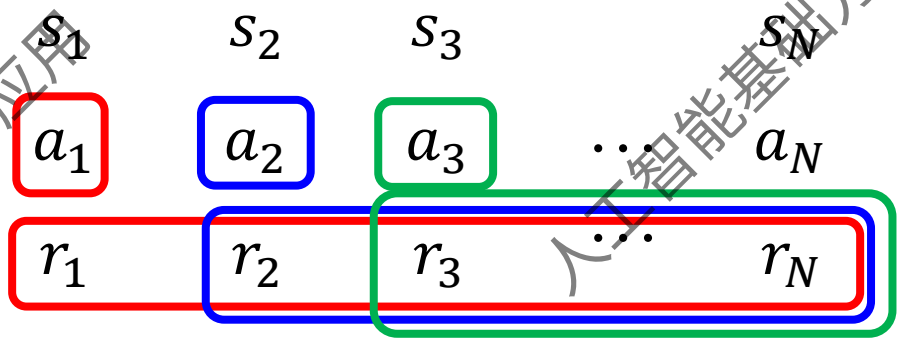
$\vdots$ $\vdots$

$\{s_N, a_N\}$ $A_N = G_N'$

$$G_t' = \sum_{n=t}^{N} \gamma^{n-t} r_n$$

人工智能基础方法与应用

智能体3-设置奖励基准



奖励的好坏是“相对的”

如果所有的奖励$r_n \geq 10$

$r_n = 10$ 是负面奖励…

设置一个奖励基准b ???

使得 G_t' 有相对的正负值

训练数据

$\{s_1, a_1\}$	$\mathrm{A}_1 = G_1' - b$
$\{s_2, a_2\}$	$\mathrm{A}_2 = G_2' - b$
$\{s_3, a_3\}$	$\mathrm{A}_3 = G_3' - b$
⋮	⋮
$\{s_N, a_N\}$	$\mathrm{A}_N = G_N' - b$

优势函数

$$G_t' = \sum_{n=t}^{N} \gamma^{n-t} r_n$$

智能体3-设置奖励基准

- 优势函数：当前动作比这个状态下的平均水平好多少

$$A(s_t, a_t) = G_t' - b$$

- 奖励基准b：在状态s_t下，按照当前策略继续行动，平均能获得多少未来累计奖励

$$b = \frac{1}{N}\sum_{i=1}^{N} G_i$$

- 在更标准的强化学习算法中，奖励基准通常不是一个常数，而是一个和状态有关的函数，叫状态价值函数

$$b(s_t) = V(s_t)$$

策略梯度

- 初始化智能体策略网络参数θ^0
- 对第 $i = 1,\ldots,T$ 次训练迭代：
  - 使用当前智能体策略θ^{i-1} 与环境交互
  - 采集多回合数据$\{s_1, a_1\}, \{s_2, a_2\}, \ldots, \{s_N, a_N\}$
  - 计算优势函数 $A_1, A_2, \ldots, A_N$
  - 构造策略梯度损失函数L（$-\bar{R}_\theta$）
  - 更新策略网络参数$\theta^i \leftarrow \theta^{i-1} - \eta\nabla L$（$\theta^{i-1} + \eta\nabla\bar{R}_\theta$）

关键点：数据采集在训练循环内部，每次策略更新后都要重新采样

策略梯度

$$\bar{R}_\theta = \sum_\tau R(\tau) p_\theta(\tau) \qquad \nabla \bar{R}_\theta = ?$$

$$\nabla \bar{R}_\theta = \sum_\tau R(\tau) \nabla p_\theta(\tau) = \sum_\tau R(\tau) p_\theta(\tau) \frac{\nabla p_\theta(\tau)}{p_\theta(\tau)}$$

$R(\tau)$ 不需要可微分，可以是黑盒

$$= \sum_\tau R(\tau) p_\theta(\tau) \nabla log p_\theta(\tau)$$

$$\nabla f(x) = f(x) \nabla log f(x)$$

$$= E_{\tau \sim p_\theta(\tau)} [R(\tau) \nabla log p_\theta(\tau)] \approx \frac{1}{N} \sum_{n=1}^{N} R(\tau^n) \nabla log p_\theta(\tau^n)$$

$$= \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} R(\tau^n) \nabla log p_\theta(a_t^n | s_t^n)$$

$$= E_{(s_t, a_t) \sim \pi_\theta} [A^\theta(s_t, a_t) \nabla log p_\theta(a_t^n | s_t^n)]$$

人工智能基础方法与应用

策略梯度

- 在监督学习中，我们知道正确答案，所以用交叉熵提高正确标签的概率

$$\nabla L_\theta = -E_\theta[\nabla log p_\theta(a|s)]$$

- 在强化学习中，我们并不知道哪个动作是正确的，只能让智能体自己去尝试。每次尝试后，环境会给出奖励反馈。如果某条轨迹获得了高奖励，我们就提高这条轨迹中动作的概率；如果奖励低，就降低这些动作的概率。

$$-\nabla \bar{R}_\theta = -E_{(s_t,a_t)\sim\pi_\theta}[A^\theta(s_t,a_t)\nabla log p_\theta(a_t^n|s_t^n)]$$

人工智能基础方法与应用

策略梯度

$$\nabla \bar{R}_\theta = E_{\tau \sim p_\theta(\tau)}[R(\tau)\nabla log p_\theta(\tau)]$$

训练数据

$\{s_1, a_1\}$ A_1

$\{s_2, a_2\}$ A_2

$\{s_3, a_3\}$ A_3

$\vdots \quad \vdots \quad \vdots$

$\{s_N, a_N\}$ A_N

$s \rightarrow$ 智能体 策略 $\theta \rightarrow a$

$$\theta^i \leftarrow \theta^{i-1} + \eta \nabla \bar{R}_\theta$$

$$\nabla \bar{R}_\theta = \frac{1}{N} \sum_{n=1}^{N} \sum_{t=1}^{T_n} R(\tau^n) \nabla log p_\theta(a_t^n | s_t^n)$$

只更新一次

每更新一次模型参数，就需要让智能体重新与环境交互，并重新采集一整批训练数据

策略梯度

- 初始化智能体策略网络参数θ^0
- 对第 $i = 1,\ldots,T$次训练迭代:
  - 使用当前智能体策略θ^{i-1}与环境交互
  - 采集多回合数据$\{s_1, a_1\}, \{s_2, a_2\}, \ldots, \{s_N, a_N\}$
  - 计算优势函数 $A_1, A_2, \ldots, A_N$
  - 构造策略梯度损失函数L
  - 更新策略网络参数$\theta^i \leftarrow \theta^{i-1} - \eta\nabla L$

θ^{i-1}的经验

可能并不适合θ^i

《主角》



第二集：
忆秦娥—刚进剧团、怯生生、不敢开口。

这个阶段收集的数据是：紧张、沉默、不会唱，也不知道怎么进入戏曲训练。

奖励：“开口”



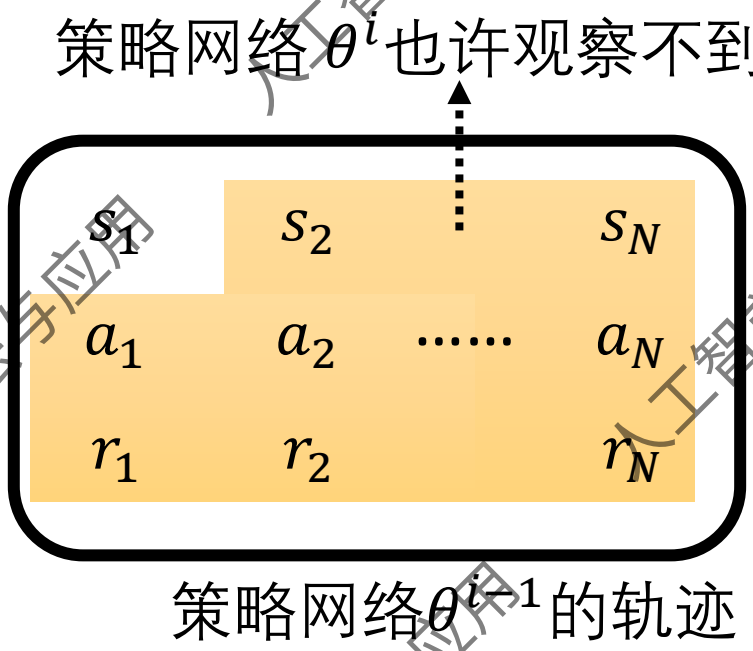
第十五集：
忆秦娥—胡三元出事、离团回乡、被调到伙房、住进库房。

这个阶段收集的数据是：主动找人学基本功，被三位老艺人看出“有韧劲”。

奖励：“成为主角”

人工智能基础方法与应用

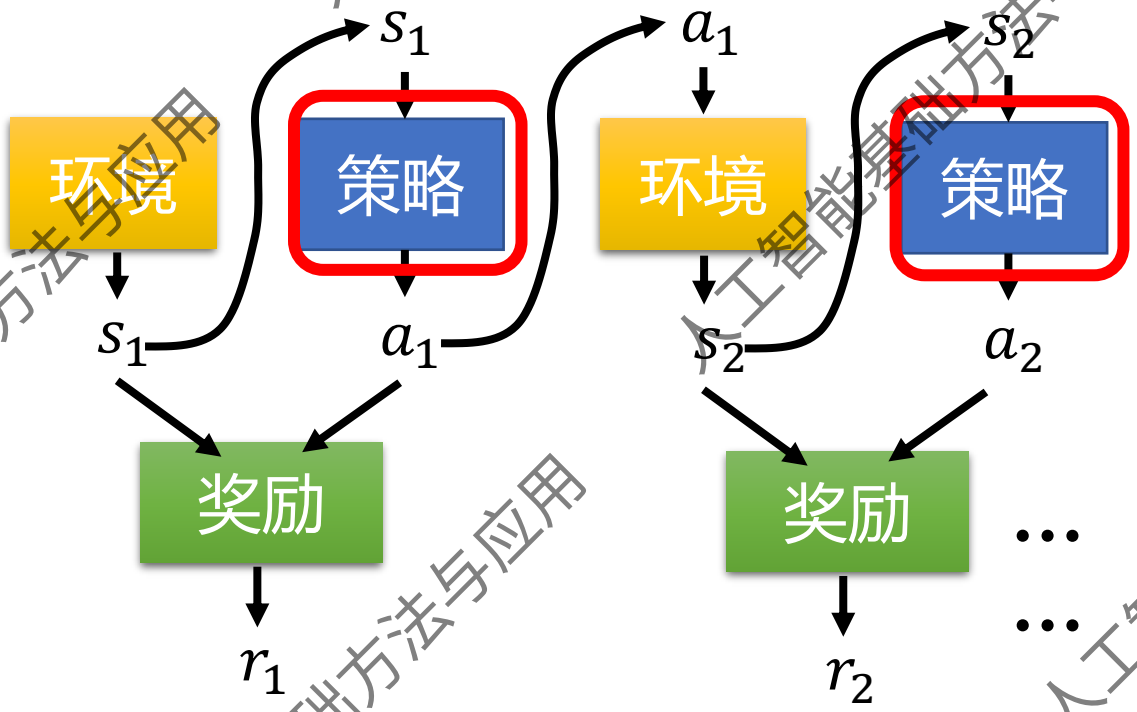
策略梯度



- 初始化智能体策略网络参数θ^0
- 对第 $\boldsymbol{i = 1,\ldots,T}$次训练迭代:
  - 使用当前智能体策略θ^{i-1} 与环境交互
  - 采集多回合数据$\{s_1, a_1\}, \{s_2, a_2\}, \ldots, \{s_N, a_N\}$
  - 计算优势函数 $A_1, A_2, \ldots, A_N$
  - 构造策略梯度损失函数L
  - 更新策略网络参数$\theta^i \leftarrow \theta^{i-1} - \eta\nabla L$

人工智能基础方法与应用

通过探索收集训练数据



…

…

假如策略总是选择 “left”

那么我们永远不会知道，如果它选择 “fire”，会发生什么

在数据采集过程中需要具有随机性

强化学习训练能否成功，很大程度上取决于探索是否充分☺

通过探索收集训练数据

- 常见的探索方法包括：
  - 随机性采样，而不是选择概率最大的动作
  - 使用 ε-贪心策略方法，一定概率随机选择动作
  - 参数或动作加入噪声，让智能体更容易尝试不同轨迹
  - 损失函数加入熵奖励项，鼓励策略分布保持一定熵值

人工智能基础方法与应用

同策略 v.s. 异策略

- 用于训练的智能体策略和用于与环境交互的智能体策略是同一个 →同策略（On-policy）
- 用于训练的智能体策略和用于与环境交互的智能体策略可以不同 →异策略 （Off-policy）

策略参数θ^{i-1}

s_1	s_2	s_3		s_N
a_1	a_2	a_3	$\cdots$	a_N
r_1	r_2	r_3	$\cdots$	r_N

用于训练新的策略参数 θ^i

避免每次更新后都重新采集交互数据

异策略

- 被**训练**的策略需要感知并修正自己与**采样**策略之间的差异



训练策略



采样策略

人工智能基础方法与应用

同策略→异策略

$$\nabla \bar{R}_\theta = E_{\tau \sim p_\theta(\tau)}[R(\tau)\nabla log p_\theta(\tau)]$$

- 使用智能体策略 π_θ 收集数据. 当 θ 更新时, 需要重新收集数据
- 目标: 使用固定策略$\pi_{\theta'}$采样，使策略θ能多次训练迭代

重要性采样

$$E_{x\sim p}[f(x)] \approx \frac{1}{N}\sum_{i=1}^{N} f(x^i)$$

x^i 从$p(x)$中采样获得

只能从$q(x)$采样得到的x^i

$$= \int f(x)p(x)dx = \int f(x)\frac{p(x)}{q(x)}q(x)dx = E_{x\sim q}[f(x)\frac{p(x)}{q(x)}]$$

重要性权重

重要性采样—方差问题

$$E_{x\sim p}[f(x)] = E_{x\sim q}[f(x)\frac{p(x)}{q(x)}]$$

$$Var_{x\sim p}[f(x)] \qquad Var_{x\sim q}[f(x)\frac{p(x)}{q(x)}]$$

$$VAR[X] = E[X^2] - (E[X])^2$$

$$Var_{x\sim p}[f(x)] = E_{x\sim p}[f(x)^2] - \left(E_{x\sim p}[f(x)]\right)^2$$

$$Var_{x\sim q}[f(x)\frac{p(x)}{q(x)}] = E_{x\sim q}\left[\left(f(x)\frac{p(x)}{q(x)}\right)^2\right] - \left(E_{x\sim q}\left[f(x)\frac{p(x)}{q(x)}\right]\right)^2$$

$$= E_{x\sim p}\left[f(x)^2\frac{p(x)}{q(x)}\right] - \left(E_{x\sim p}[f(x)]\right)^2$$

同策略→异策略

$$\nabla \bar{R}_\theta = E_{\tau \sim p_\theta(\tau)}[R(\tau)\nabla log p_\theta(\tau)]$$

- 使用智能体策略 π_θ 收集数据. 当 θ 更新时, 需要重新收集数据
- 目标:使用固定策略$\pi_{\theta'}$采样，使策略θ能多次训练迭代

$$\nabla \bar{R}_\theta = E_{\tau \sim p_{\theta'}(\tau)}\left[\frac{p_\theta(\tau)}{p_{\theta'}(\tau)} R(\tau)\nabla log p_\theta(\tau)\right]$$

- 依据采样策略θ'大量采集数据
- 使用该数据可以多次训练θ

重要性采样 $E_{x \sim p}[f(x)] = E_{x \sim q}[f(x)\frac{p(x)}{q(x)}]$

异策略目标函数求解

梯度更新

$$\nabla f(x) = f(x)\nabla log f(x)$$

$$\nabla \bar{R}_\theta = E_{(s_t,a_t)\sim\pi_\theta}[A^\theta(s_t,a_t)\nabla log p_\theta(a_t^n|s_t^n)]$$

$A^{\theta'}(s_t,a_t)$ 从采样数据中获得

$$= E_{(s_t,a_t)\sim\pi_{\theta'}}[\frac{P_\theta(s_t,a_t)}{P_{\theta'}(s_t,a_t)} A^\theta(s_t,a_t)\nabla log p_\theta(a_t^n|s_t^n)]$$

$$= E_{(s_t,a_t)\sim\pi_{\theta'}}[\frac{p_\theta(a_t|s_t)}{p_{\theta'}(a_t|s_t)} \frac{p_\theta(s_t)}{p_{\theta'}(s_t)} A^{\theta'}(s_t,a_t)\nabla log p_\theta(a_t^n|s_t^n)]$$

$$J^{\theta'}(\theta) = E_{(s_t,a_t)\sim\pi_{\theta'}}\left[\frac{p_\theta(a_t|s_t)}{p_{\theta'}(a_t|s_t)} A^{\theta'}(s_t,a_t)\right]$$

p_θ和$p_{\theta'}$不能差别太大

PPO / TRPO

θ 参数可以和θ'参数差别很大

限制策略行为的变化，而不是直接限制网络参数的变化

近端策略优化（Proximal Policy Optimization，PPO）

$$J_{PPO}^{\theta'}(\theta) = J^{\theta'}(\theta) - \beta KL(\theta, \theta')$$

$$\nabla f(x) = f(x)\nabla log f(x)$$

$$J^{\theta'}(\theta) = E_{(s_t,a_t)\sim\pi_{\theta'}}\left[\frac{p_\theta(a_t|s_t)}{p_{\theta'}(a_t|s_t)}A^{\theta'}(s_t,a_t)\right]$$

信赖域策略优化（Trust Region Policy Optimization，TRPO)

$$J_{TRPO}^{\theta'}(\theta) = E_{(s_t,a_t)\sim\pi_{\theta'}}\left[\frac{p_\theta(a_t|s_t)}{p_{\theta'}(a_t|s_t)}A^{\theta'}(s_t,a_t)\right]$$

$$\text{s.t.}\ KL(\theta, \theta') < \delta$$

PPO算法

$$J^{\theta^k}(\theta) \approx \sum_{(s_t,a_t)} \frac{p_\theta(a_t|s_t)}{p_{\theta^k}(a_t|s_t)} A^{\theta^k}(s_t,a_t)$$

- 初始化策略参数θ^0
- 第k=1,…,K次迭代
  - 使用 θ^k 与环境交互获得$\{s_t,a_t\}$ 并计算优势函数$A^{\theta^k}(s_t,a_t)$
  - 寻找 θ 以优化 $J_{PPO}(\theta)$

$$J_{PPO}^{\theta^k}(\theta) = J^{\theta^k}(\theta) - \beta KL(\theta,\theta^k)$$

多次更新参数

- If $KL(\theta,\theta^k) > KL_{max}$, 增加 β
- If $KL(\theta,\theta^k) < KL_{min}$, 减小 β

自适应KL惩罚

人工智能基础方法与应用

DeepMind - PPO



人工智能基础方法与应用

强化学习的基本概念2

序号	概念	定义
10	奖励延迟（Reward Delay）	当前动作的效果可能在未来才体现
11	折扣因子（Discount Factor）	控制智能体重视未来奖励的程度
12	同策略（On-policy）	用当前策略采样，并用这些数据训练当前策略
13	异策略（Off-policy）	用其他策略或旧策略采集的数据训练当前策略
14	近端策略优化（PPO）	限制策略更新幅度，提高训练稳定性
15	探索（Exploration）	通过尝试不同动作发现更优行为

人工智能基础方法与应用

强化学习：在互动中学习

机器学习 ≈ 寻找从输入到输出的映射函数



智能体

动作 = f(观测)

观测

映射函数
输入

动作

映射函数
输出

寻找**策略函数f**

以最大化长期累积奖励

奖励



环境

人工智能基础方法与应用

强化学习的基本概念

序号	概念	定义
1	智能体（Agent）	强化学习中的学习主体
2	环境（Environment）	智能体所处的外部世界
3	观测（Observation）	从环境中获得的信息，用于描述当前状态
4	动作（Action）	根据当前观测做出的操作
5	回合（Episode）	从初始观测直到任务结束的完整交互过程
6	轨迹（Trajectory）	一个回合中，产生的状态、动作序列
7	策略（Policy）	行为规则，决定在某个观测下应该采取什么动作
8	奖励（Reward）	执行一次动作后，环境给出的立即标量反馈
9	累积奖励（Return）	一个回合中，所有即时奖励的总和
10	奖励延迟（Reward Delay）	当前动作的效果可能在未来才体现
11	折扣因子（Discount Factor）	控制智能体重视未来奖励的程度
12	同策略（On-policy）	用当前策略采样，并用这些数据训练当前策略
13	异策略（Off-policy）	用其他策略或旧策略采集的数据训练当前策略
14	近端策略优化（PPO）	限制策略更新幅度，提高训练稳定性
15	探索（Exploration）	通过尝试不同动作发现更优行为

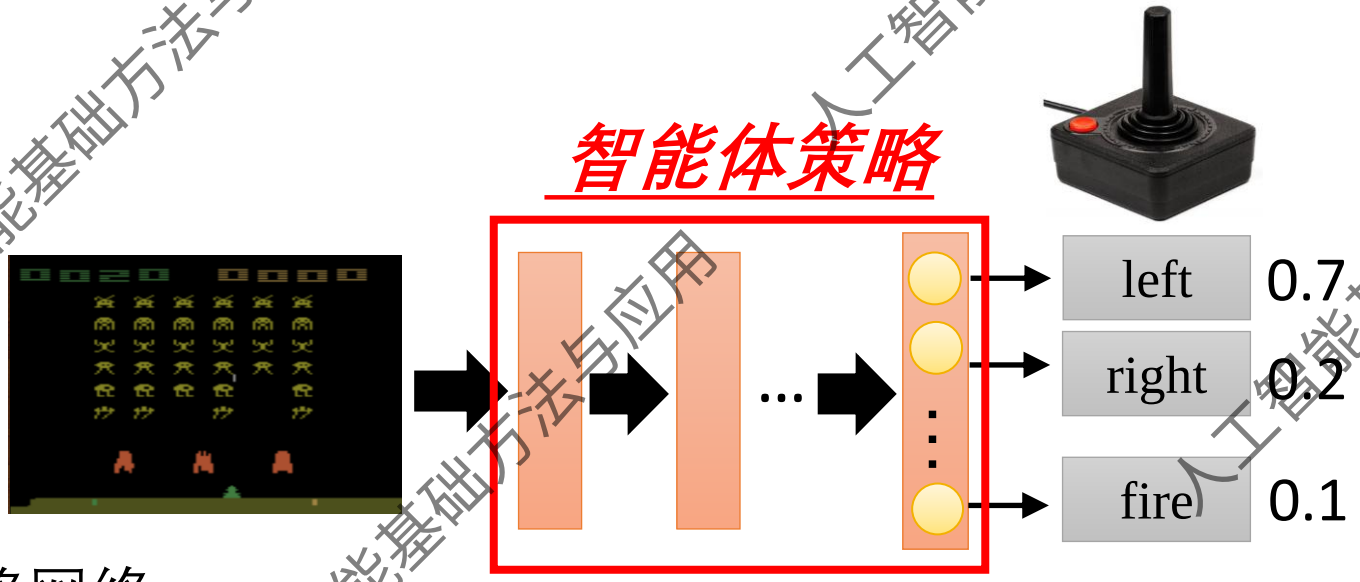
人工智能基础方法与应用

策略梯度

- 初始化智能体策略网络参数θ^0
- 对第 ***i = 1,…,T*** 次训练迭代：
  - 使用当前智能体策略θ^{i-1} 与环境交互
  - 采集多回合数据$\{s_1, a_1\}, \{s_2, a_2\}, \ldots, \{s_N, a_N\}$
  - 计算优势函数 $A_1, A_2, \ldots, A_N$
  - 构造策略梯度损失函数（$-\bar{R}_\theta$）
  - 更新策略网络参数$\theta^i \leftarrow \theta^{i-1} - \eta\nabla L$（$\theta^{i-1} + \eta\nabla\bar{R}_\theta$）

人工智能基础方法与应用

例子



1. 初始化策略网络

$$\pi_{\theta^0}(a_L|s) = 0.7 \qquad \pi_{\theta^0}(a_R|s) = 0.2 \qquad \pi_{\theta^0}(a_F|s) = 0.1$$

2. 使用当前策略与环境交互

$$(s, a_L, r = 0) \qquad (s, a_R, r = 0) \qquad (s, a_F, r = 5)$$

3. 计算优势函数

计算奖励基准 $b = \dfrac{0+0+5}{3} = \dfrac{5}{3}$

人工智能基础方法与应用

计算优势函数

$$A_L = 0 - \frac{5}{3} = -\frac{5}{3}$$

$$A_R = 0 - \frac{5}{3} = -\frac{5}{3}$$

$$A_F = 5 - \frac{5}{3} = \frac{10}{3}$$

可以看到:

$$A_L < 0 \qquad A_R < 0 \qquad A_F > 0$$

4. 构造策略梯度损失函数 $L = -\bar{R}_\theta = -\frac{1}{N}\sum_{n=1}^{N}\sum_{t=1}^{T_n} A^\theta(s_t, a_t) log\pi_\theta(a_t^n|s_t^n)$

$$L(\theta) = -\frac{1}{3}\left[A_L \log \pi_\theta(a_L|s) + A_R \log \pi_\theta(a_R|s) + A_F \log \pi_\theta(a_F|s)\right]$$

5. 计算策略梯度

$$\nabla_\theta \bar{R}_\theta\big|_{\theta=\theta^0} \approx \frac{1}{3}\left[-\frac{5}{3}\nabla_\theta \log \pi_\theta(a_L|s) - \frac{5}{3}\nabla_\theta \log \pi_\theta(a_R|s) + \frac{10}{3}\nabla_\theta \log \pi_\theta(a_F|s)\right]_{\theta=\theta^0}$$

6.更新策略参数

$$\theta^1 = \theta^0 + \eta \nabla_\theta \bar{R}_{\theta^0}$$

7. 更新后的动作概率

$$\eta = 0.1$$

$$\pi_{\theta^1}(a_L|s) \approx 0.688$$

$$\pi_{\theta^1}(a_R|s) \approx 0.196$$

$$\pi_{\theta^1}(a_F|s) \approx 0.116$$

动作	更新前概率	更新后概率	变化	优势函数
向左	0.700	0.688	降低	$-\frac{5}{3}$
向右	0.200	0.196	降低	$-\frac{5}{3}$
开火	0.100	0.116	提高	$\frac{10}{3}$

人工智能基础方法与应用

目录

RL是什么？（ML的三步骤）

策略梯度

演员-评论员

稀疏奖励

无奖励：从演示中学习

人工智能基础方法与应用

演员（Actor）

- 负责“做动作”的智能体策略 $\pi_\theta(a|s)$
- 定义：根据当前状态决定以多大概率执行候选动作
- 策略梯度就是训练Actor的方法：
  - 如果某个动作带来较高回报，就提高它的概率
  - 如果某个动作带来较差结果，就降低它的概率
- 核心思想：Actor先去环境中尝试动作，再根据奖励反馈调整自己的策略
- 问题：只靠Actor独立试错，训练往往不稳定

第一次游戏：$\Delta\theta^{(1)} \propto 15\nabla_\theta \log \pi_\theta(\text{开火} \mid s_t)$

第二次游戏：$\Delta\theta^{(2)} \propto -9\nabla_\theta \log \pi_\theta(\text{开火} \mid s_t)$

评论员（Critic） $G_1' = r_1 + \gamma r_2 + \gamma^2 r_3 + \cdots\cdots$

- 评论员：在给定智能体策略θ的情况下，当智能体观测到状态s (或进一步采取动作a)时，接下来能获得多少奖励
- 价值函数$V^\theta(s)$，也叫状态价值函数：表示在使用智能体策略θ，得到状态 s 后，预计能够获得的折扣累计奖励G





$V^\theta(s)$ 较大



$V^\theta(s)$ 较小

评论员的输出值取决于它正在评估的智能体策略

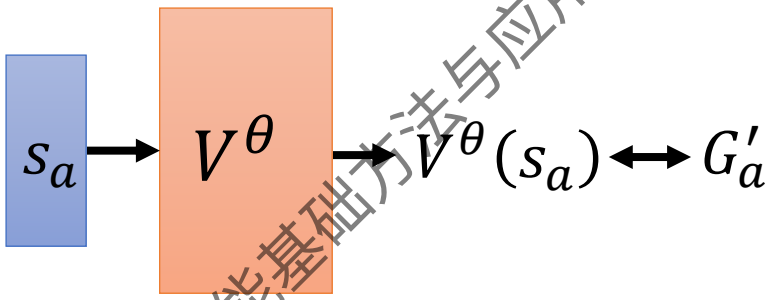
如何估计 $V^{\theta}(s)$-方法1

- 基于蒙特卡洛的方法 **Monte-Carlo (MC)**

评论员通过观察智能体θ与环境交互的过程来学习评估状态价值

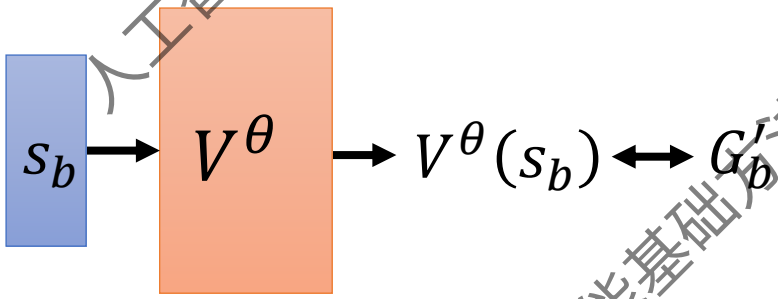
看到状态s_a之后，

直到该回合结束，所获得的累计奖励为 G'_a



看到状态s_b之后，

直到该回合结束，所获得的累计奖励为G'_b



为什么叫“蒙特卡洛方法”？

- 数学家乌拉姆在思考纸牌接龙的获胜概率时发现：
  - 与其推导所有可能情况，不如随机模拟大量牌局，用成功次数的比例估计真实概率
- 冯·诺依曼将这一思想用于中子扩散等复杂计算问题
- 梅特罗波利斯将这种统计计算方法命名为“Monte Carlo”，名称来自以赌场闻名的摩纳哥蒙特卡洛

当一个期望值难以直接计算时，即通过多次**随机采样**，并对采样结果**取平均**，近似真实期望



如何估计 $V^{\theta}(s)$-方法2

- 时序差分方法 **Temporal-difference (TD)**

$\cdots s_t, a_t, r_t, s_{t+1} \cdots$

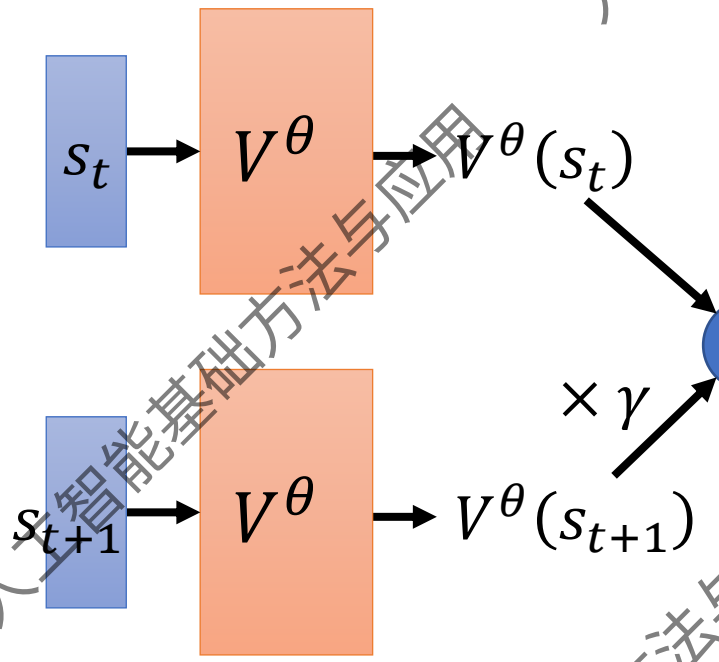
(这里暂时忽略期望项)

$$V^{\theta}(s_t) = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} \cdots$$

$$V^{\theta}(s_{t+1}) = r_{t+1} + \gamma r_{t+2} + \cdots$$

$$\boxed{V^{\theta}(s_t) = \gamma V^{\theta}(s_{t+1}) + r_t}$$

贝尔曼方程



$$V^{\theta}(s_t) - \gamma V^{\theta}(s_{t+1}) \leftrightarrow r_t$$

在某些任务中，一个回合可能很长。若必须等到回合结束后才更新模型，学习效率会非常低

自举（Bootstrapping）

- 初始含义：靠自己的鞋带把自己拉起来
- 引申含义：利用系统自己当前已有的结果，进一步改进自己
- 在更新当前状态价值时，不等待真实的完整回报，而是把下一状态当前的价值估计，当作训练目标的一部分
- TD 实际上是用已有的估计$V^{\theta}(s_{t+1})$，帮助更新另一个估计 $V^{\theta}(s_t)$

人工智能基础方法与应用

$V^{\theta}(s)$如何保证估计准确

- 初始的$V^{\theta}(s)$是随机的，环境给出的奖励r是真实观测

某一回合最后一步获得奖励10，并进入终止状态：

$$V(s_{\text{terminal}}) = 0$$

因此倒数第二个状态的目标是：

$$y_t = r_t + \gamma V(s_{\text{terminal}}) = 10$$

这时网络就获得了一个比较可靠的训练目标。随后，这个信息可以逐步向前传播

$$V(s_{t-1}) \approx r_{t-1} + \gamma V(s_t)$$

即使初始价值是随机的，终止奖励和实际观测奖励会像“锚点”一样，逐步把价值估计拉到合理位置

TD 方法如何逐渐修正随机估计

- TD 误差: $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$

  初始时: $V(s_t) = 0.3, \quad V(s_{t+1}) = 0.5$

  实际获得: $r_t = 2, \quad \gamma = 0.9$

  那么 TD 目标是: $y_t = 2 + 0.9 \times 0.5 = 2.45$

  TD 误差为: $\delta_t = 2.45 - 0.3 = 2.15$

经过大量状态转移样本后，正负误差会不断修正网络输出

人工智能基础方法与应用

MC v.s. TD

- 评论员已经观察到了以下 8 个回合：
  - $s_a, r = 0, s_b, r = 0$, END
  - $s_b, r = 1$, END
  - $s_b, r = 1$, END
  - $s_b, r = 1$, END
  - $s_b, r = 1$, END
  - $s_b, r = 1$, END
  - $s_b, r = 1$, END
  - $s_b, r = 0$, END

(假设$\gamma = 1$，并且这里暂时忽略动作的影响)

$$V^{\theta}(s_b) = 3/4$$

$$V^{\theta}(s_a) = ?$$ 0? 3/4?

蒙特卡洛: $V^{\theta}(s_a) = 0$

时序差分:

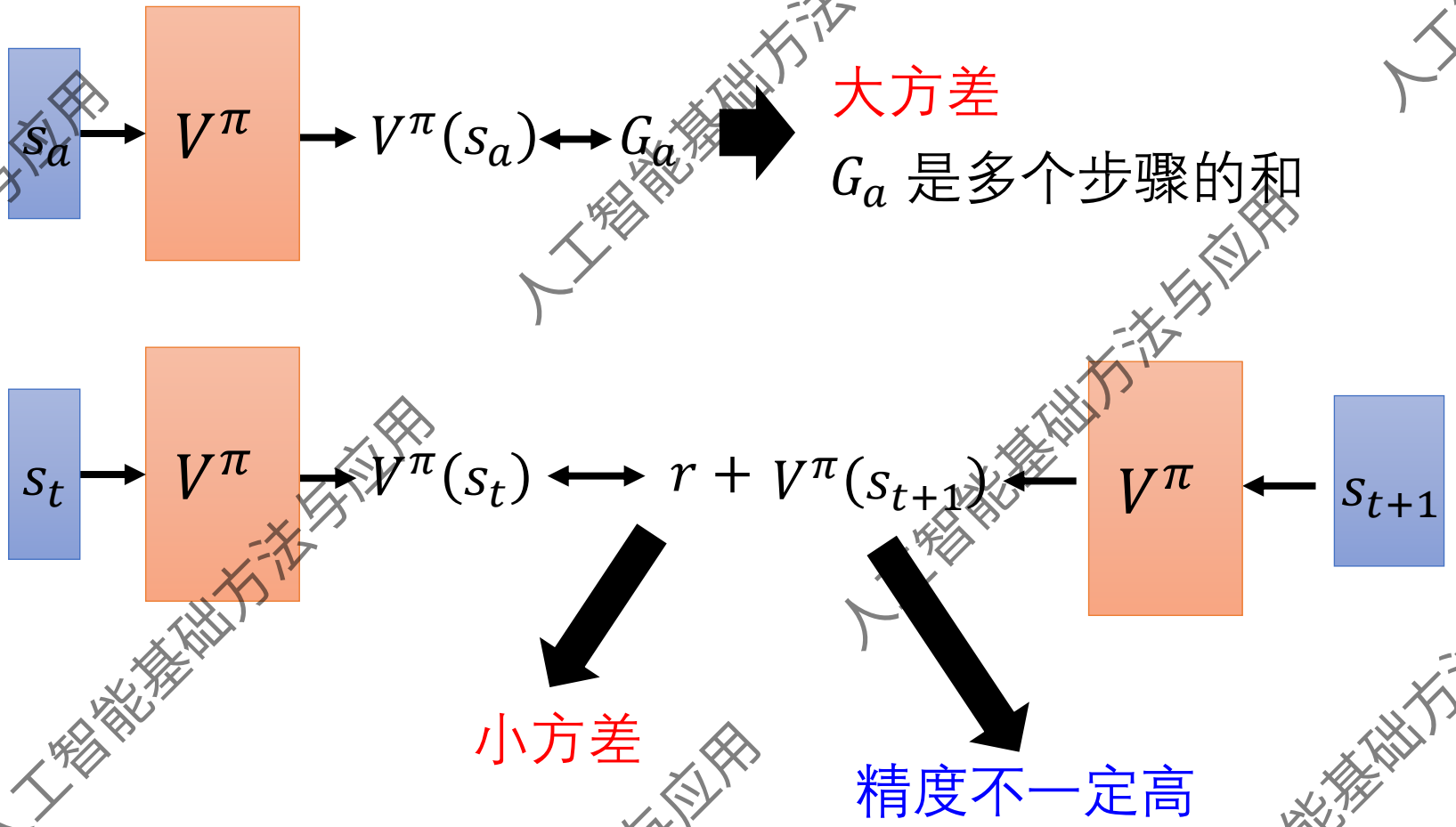
$$V^{\theta}(s_a) = V^{\theta}(s_b) + r$$

3/4 3/4 0

折扣因子严重影响累计奖励中不同时间步奖励的方差

$$Var[kX] = k^2 Var[X]$$

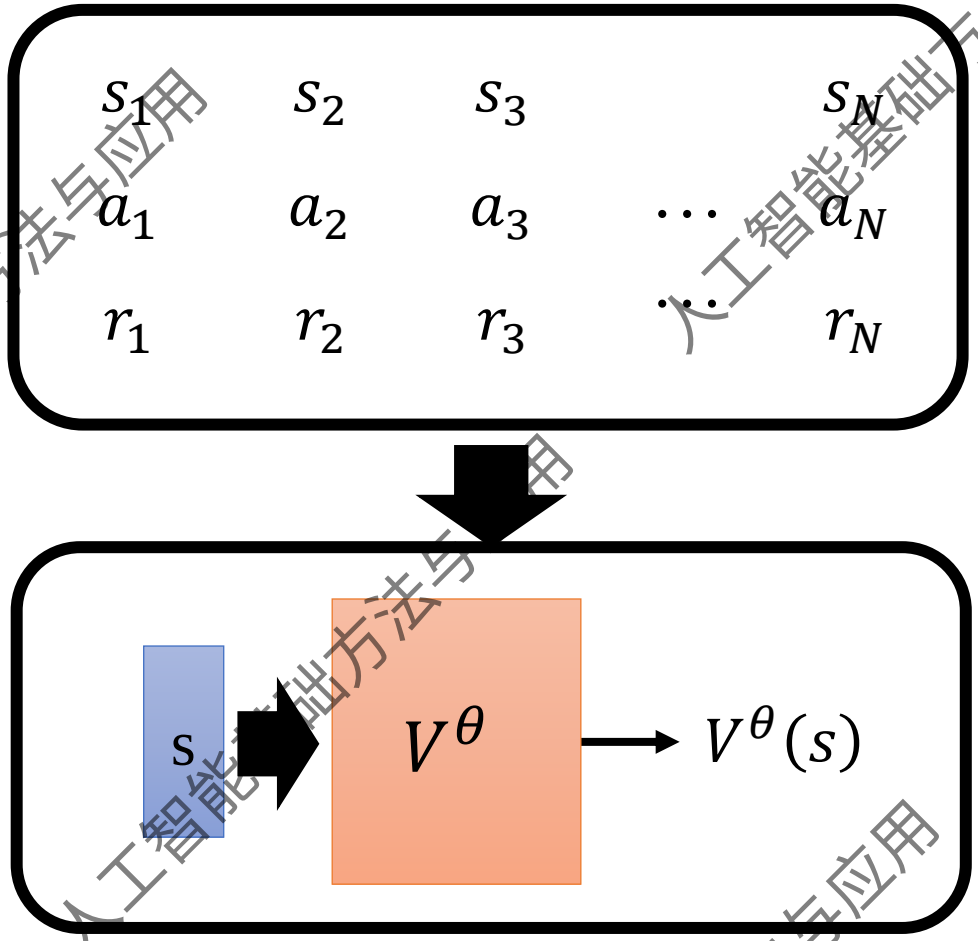
MC v.s. TD



人工智能基础方法与应用

智能体3.5

$-b$



Training Data

$\{s_1, a_1\}$ $A_1 = G_1' - b$

$\{s_2, a_2\}$ $A_2 = G_2' - b$

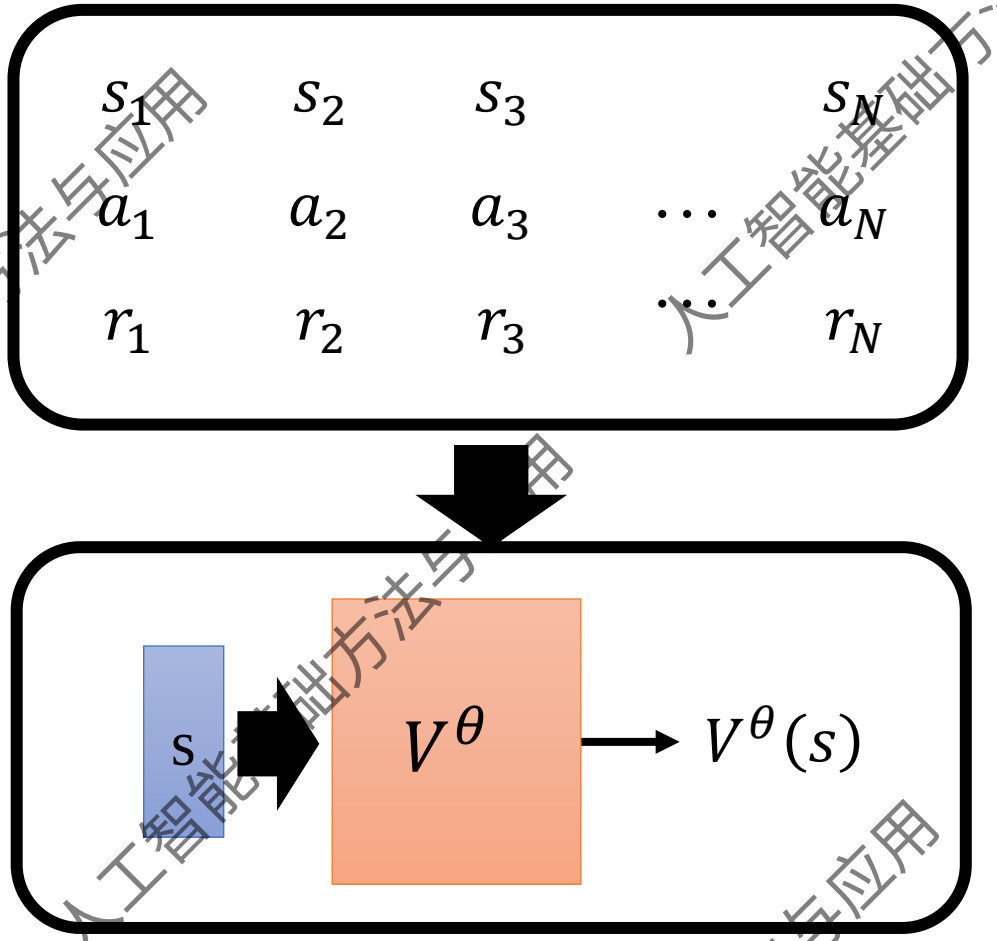
$\{s_3, a_3\}$ $A_3 = G_3' - b$

$\vdots$ $\vdots$ $\vdots$

$\{s_N, a_N\}$ $A_N = G_N' - b$

人工智能基础方法与应用

智能体3.5

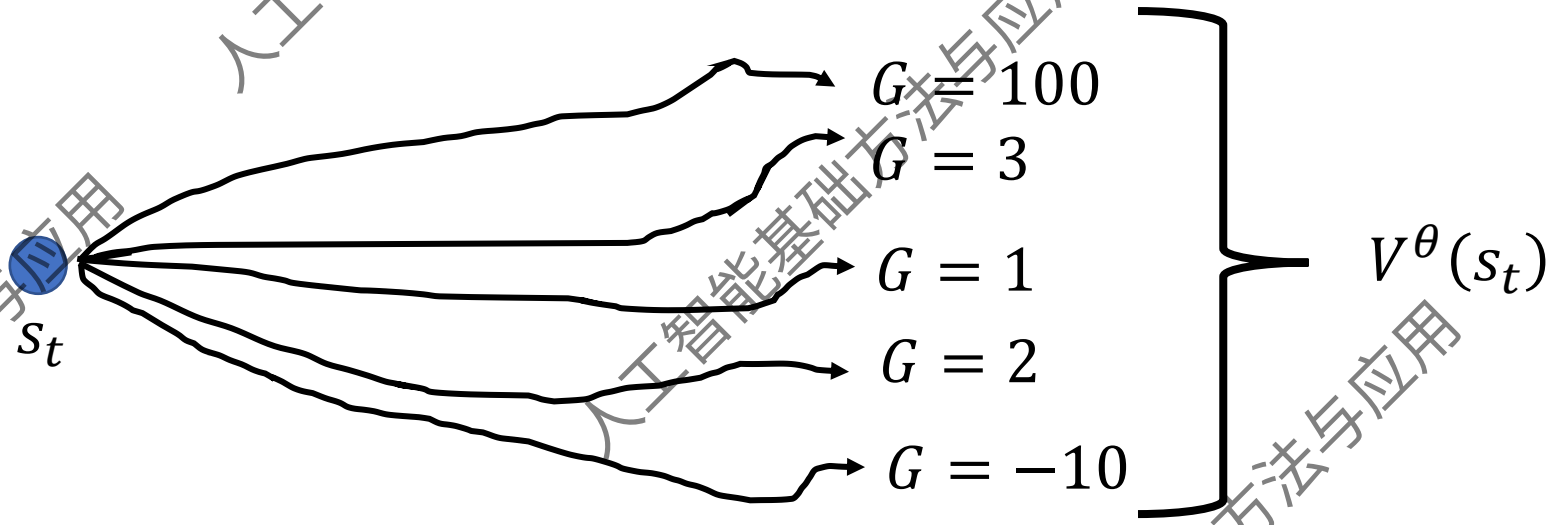


Training Data

$$\{s_1, a_1\} \quad A_1 = G_1 - V^\theta(s_1)$$
$$\{s_2, a_2\} \quad A_2 = G_2' - V^\theta(s_2)$$
$$\{s_3, a_3\} \quad A_3 = G_3' - V^\theta(s_3)$$
$$\vdots \quad \vdots \quad \vdots$$
$$\{s_N, a_N\} \quad A_N = G_N' - V^\theta(s_N)$$

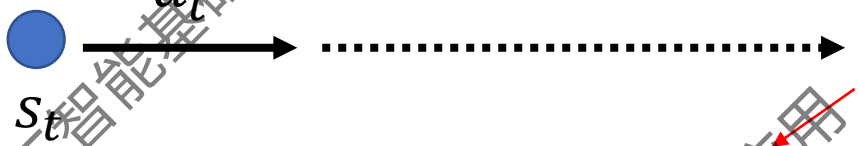
智能体3.5

$\{s_t, a_t\}$ $\mathrm{A}_t = G'_t - V^\theta(s_t)$



(当前状态下存在多种可能动作，不一定只采取当前采样到的动作a_t)

$\mathrm{A}_t > 0$ a_t优于平均水平

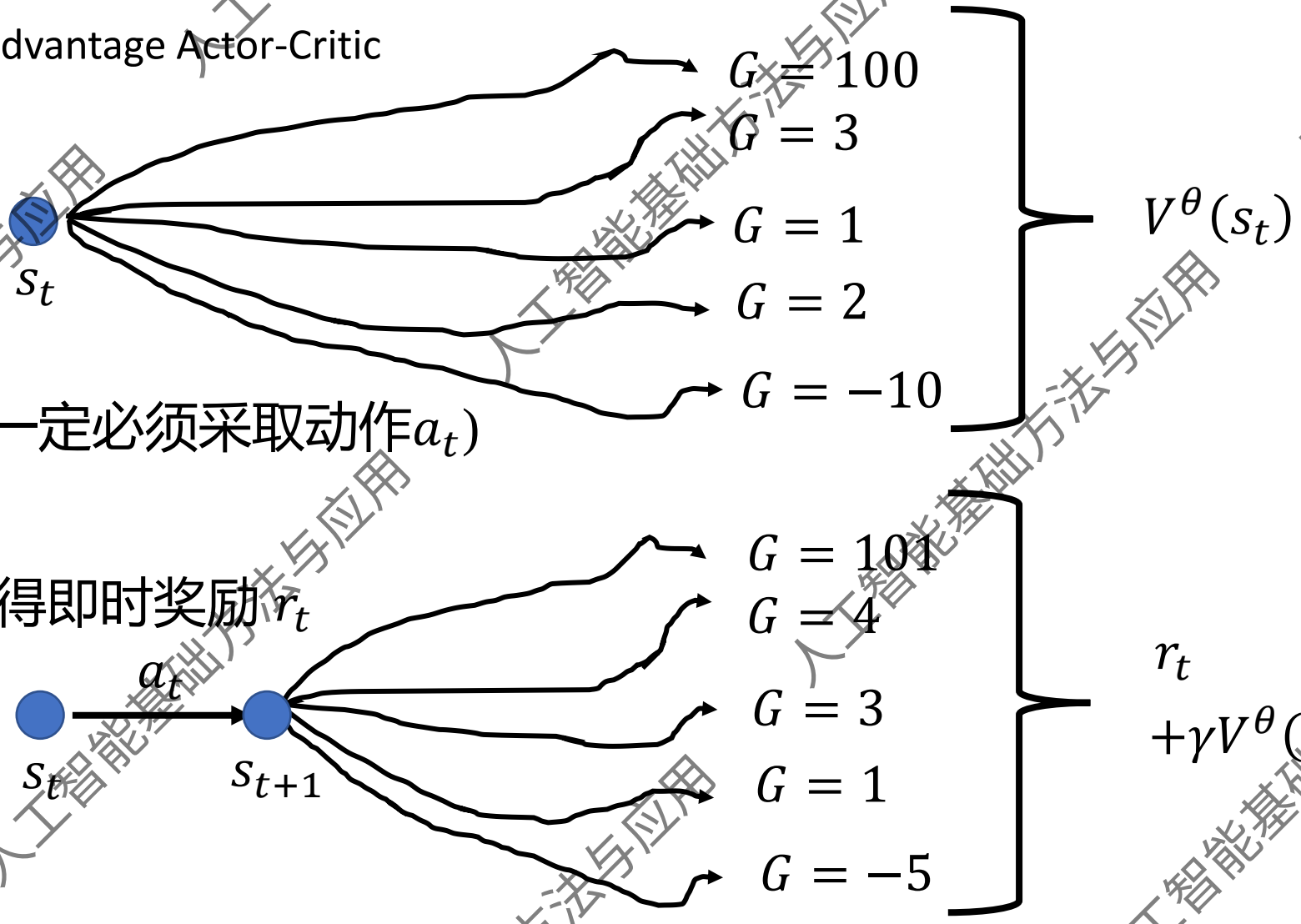


$\mathrm{A}_t < 0$ a_t低于平均水平

只是一次采样结果

智能体4-优势演员-评论员方法

Advantage Actor-Critic

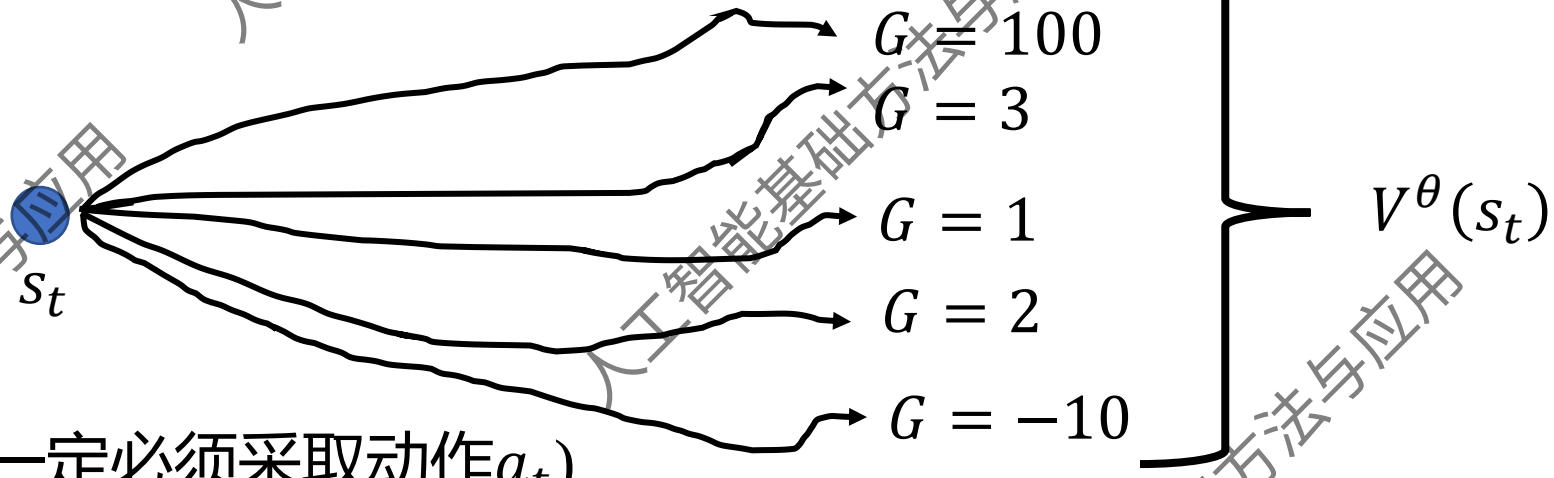


人工智能基础方法与应用

智能体4

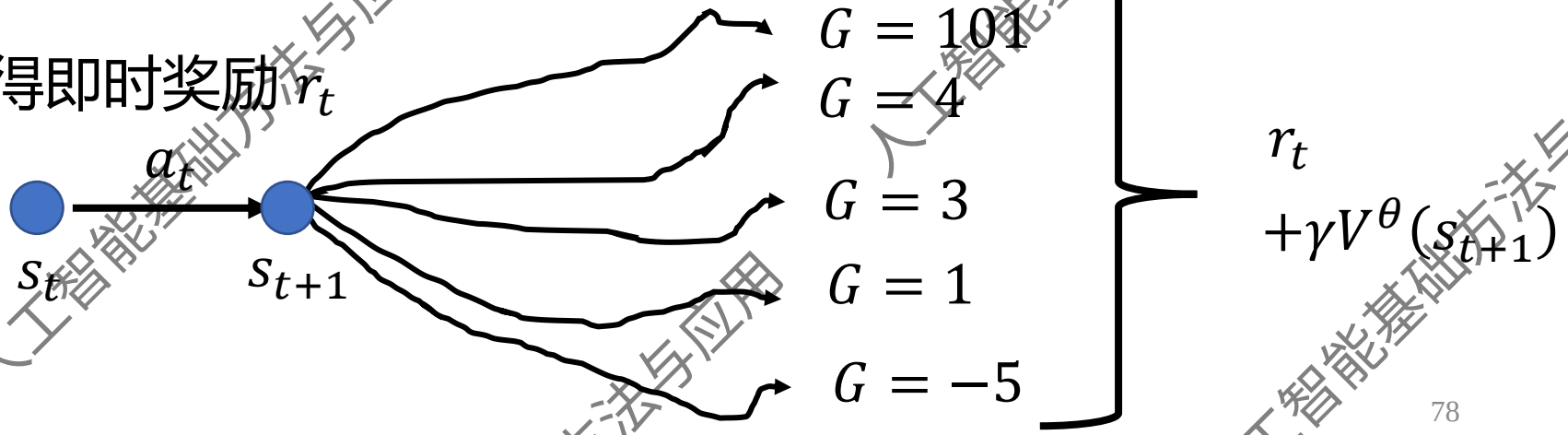
$$r_t + \gamma V^{\theta}(s_{t+1}) - V^{\theta}(s_t)$$

$\{s_t, a_t\}$ $A_t = G'_t - V^{\theta}(s_t)$



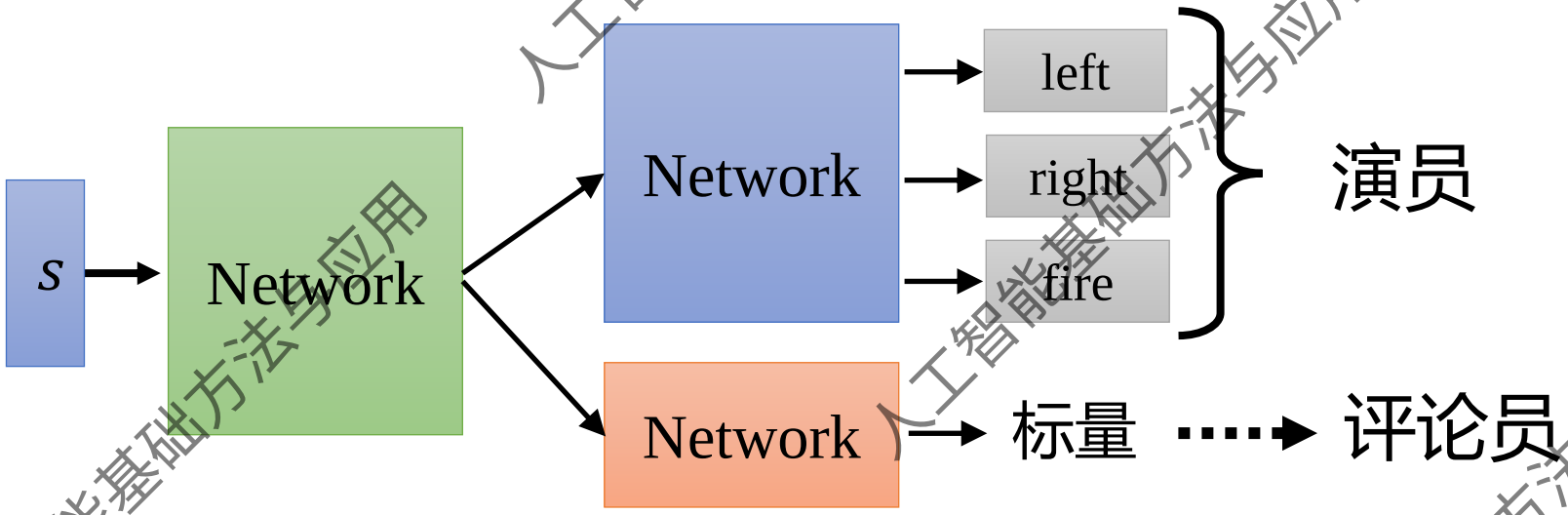
(不一定必须采取动作a_t)

获得即时奖励 r_t



演员-评论员方法技巧

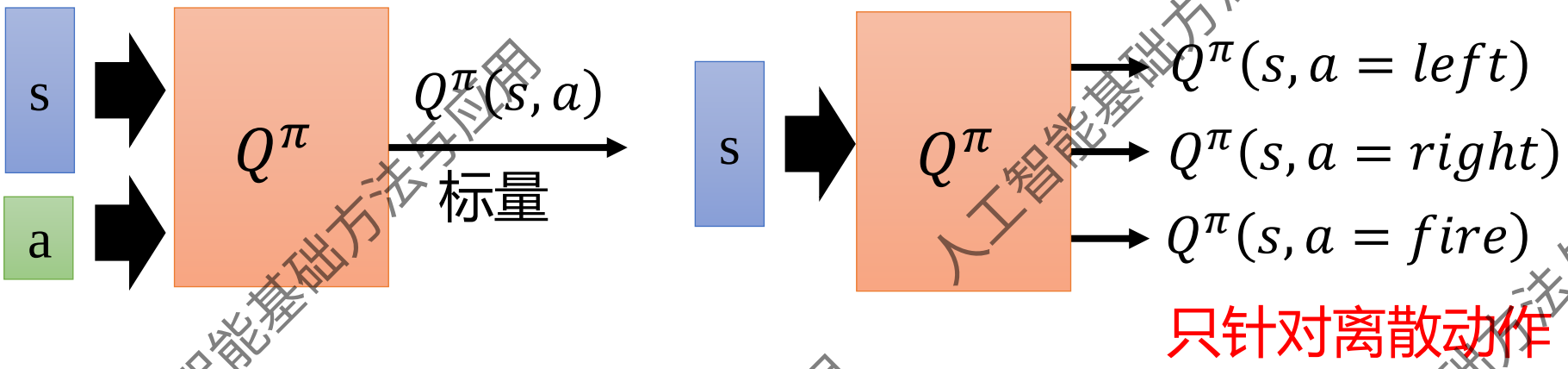
- 演员网络和评论员网络可以共享部分参数



人工智能基础方法与应用

评论员2-状态动作值函数

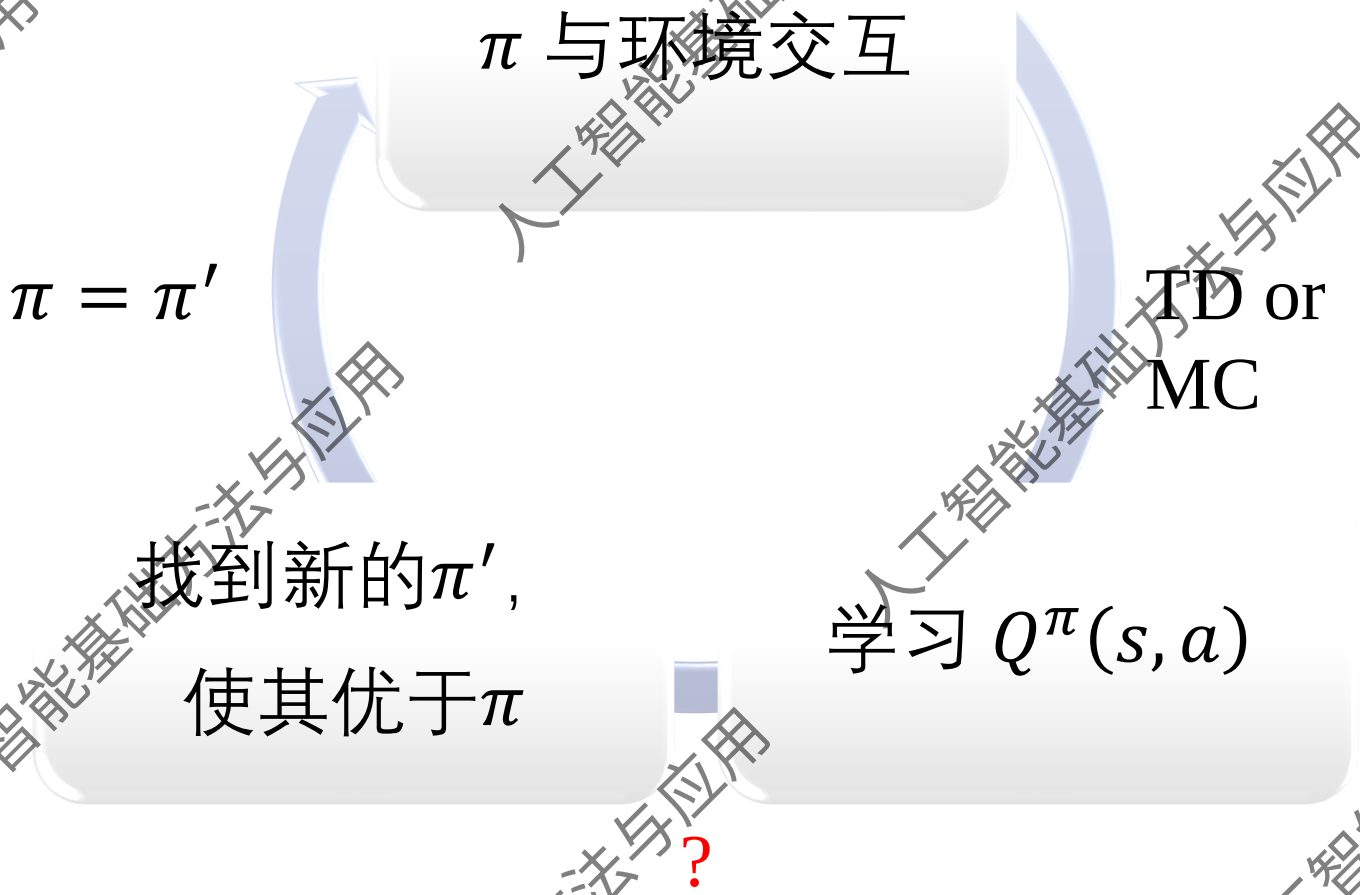
- 状态–动作值函数$Q^{\pi}(s,a)$，也称动作价值函数或Q函数
  - 表示智能体在状态 s下采取动作a后，继续按照策略π行动时，预期能够获得的累计奖励



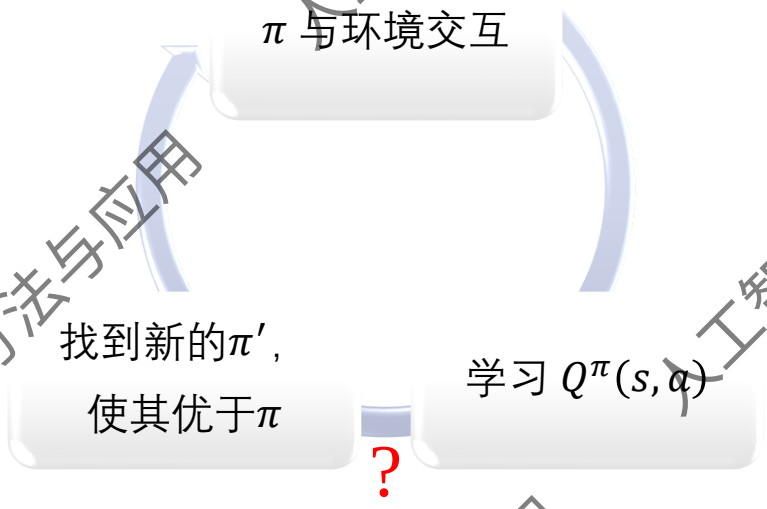
人工智能基础方法与应用

Q-Learning

- 通过学习每个状态-动作对的长期价值，并选择 Q 值最大的动作，实现策略的逐步改进



Q-Learning



- 已知 $Q^{\pi}(s,a)$的情况下，找到新策略π' 好于旧策略π
  - “怎么好”：$V^{\pi'}(s) \geq V^{\pi}(s)$，对于所有状态s

$$\pi'(s) = arg\max_{a} Q^{\pi}(s,a)$$

- π' 没有额外的参数，只依赖Q函数
- 该方法不适合处理连续动作空间

Q-Learning

$$\pi'(s) = arg\max_a Q^{\pi}(s,a)$$

$V^{\pi'}(s) \geq V^{\pi}(s)$,, 对于所有状态s

$$V^{\pi}(s) = Q^{\pi}\big(s,\pi(s)\big)$$

$$\leq \max_a Q^{\pi}(s,a) = Q^{\pi}\big(s,\pi'(s)\big)$$

$$V^{\pi}(s) \leq Q^{\pi}\big(s,\pi'(s)\big)$$

$$= E[r_{t+1} + V^{\pi}(s_{t+1})|s_t = s, a_t = \pi'(s_t)]$$

$$\leq E[r_{t+1} + Q^{\pi}\big(s_{t+1},\pi'(s_{t+1})\big)|s_t = s, a_t = \pi'(s_t)]$$

$$= E[r_{t+1} + r_{t+2} + V^{\pi}(s_{t+2})|\dots]$$

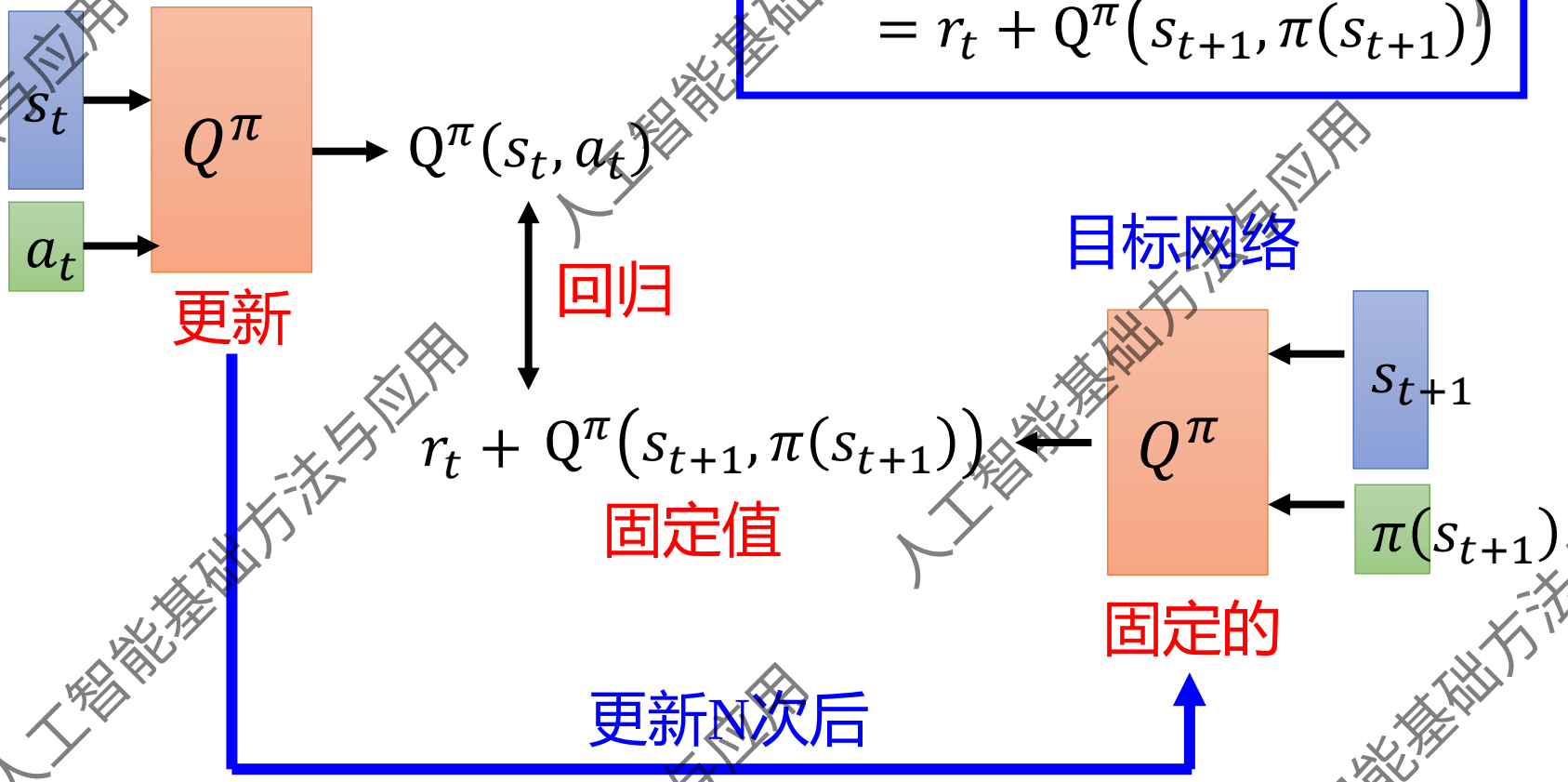
$$\leq E[r_{t+1} + r_{t+2} + Q^{\pi}\big(s_{t+2},\pi'(s_{t+2})\big)|\dots] \dots \leq V^{\pi'}(s)$$

人工智能基础方法与应用

目标网络

$$\cdots s_t, a_t, r_t, s_{t+1} \cdots$$

$$Q^{\pi}(s_t, a_t) = r_t + Q^{\pi}\left(s_{t+1}, \pi(s_{t+1})\right)$$



目标网络

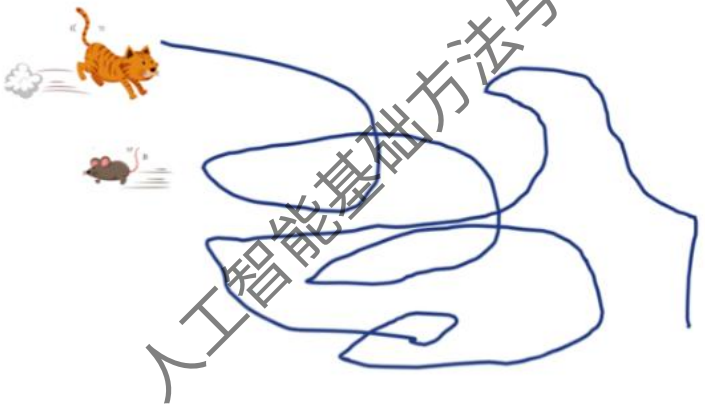


Q目标　　Q估计

(a) 猫追老鼠



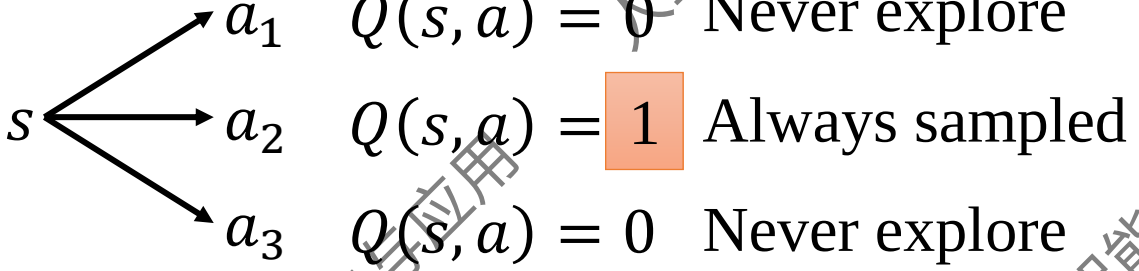
(b) 猫和老鼠都在动



(c) 猫和老鼠的优化轨迹

让正在学习的Q网络追赶一个相对稳定的目标，
而不是追赶一个每一步都在快速移动的目标

探索



- 策略由Q函数决定

$$a = arg\max_a Q(s,a)$$

利用：智能体只选择当前最好的动作。这种方式不利于数据采集

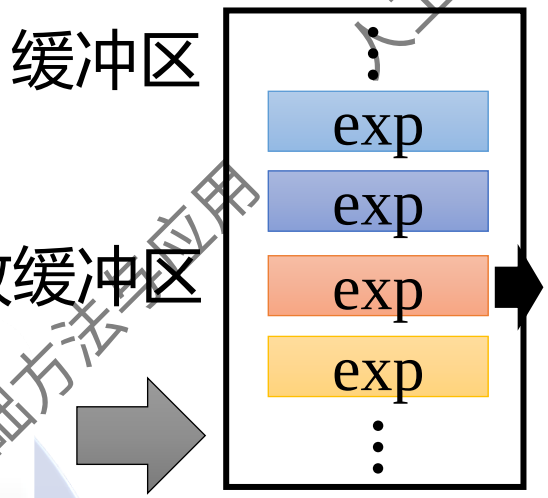
ε-贪心策略 在学习过程中，ε通常会逐渐减小

$$a = \begin{cases} arg\max_a Q(s,a), & 以1-\varepsilon概率 \\ random, & otherwise \end{cases}$$

玻尔兹曼探索

$$P(a|s) = \frac{exp\big(Q(s,a)\big)}{\sum_a exp\big(Q(s,a)\big)}$$

回放缓冲区



将经验存入回放缓冲区

回放缓冲区中的数据可能来自不同阶段的策略

如果经验池已满，就丢弃较早的旧经验

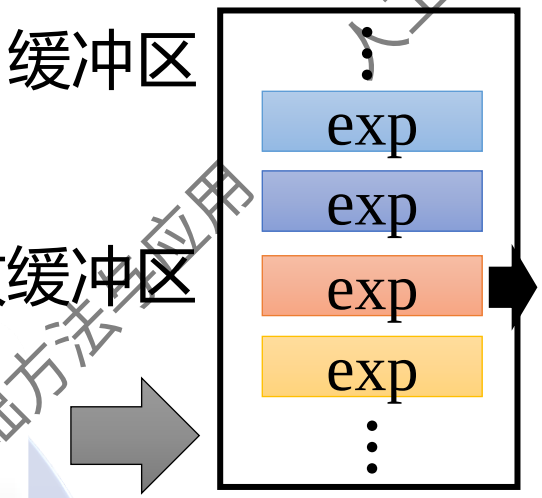
π 与环境交互

学习 $Q^{\pi}(s,a)$

找到新的π'，使其优于π

$\pi = \pi'$

回放缓冲区



将经验存入回放缓冲区

π 与环境交互

在每次迭代中:

1.从经验池中采样一批数据

2. 更新Q函数

学习 $Q^{\pi}(s,a)$

找到新的π'，使其好于π

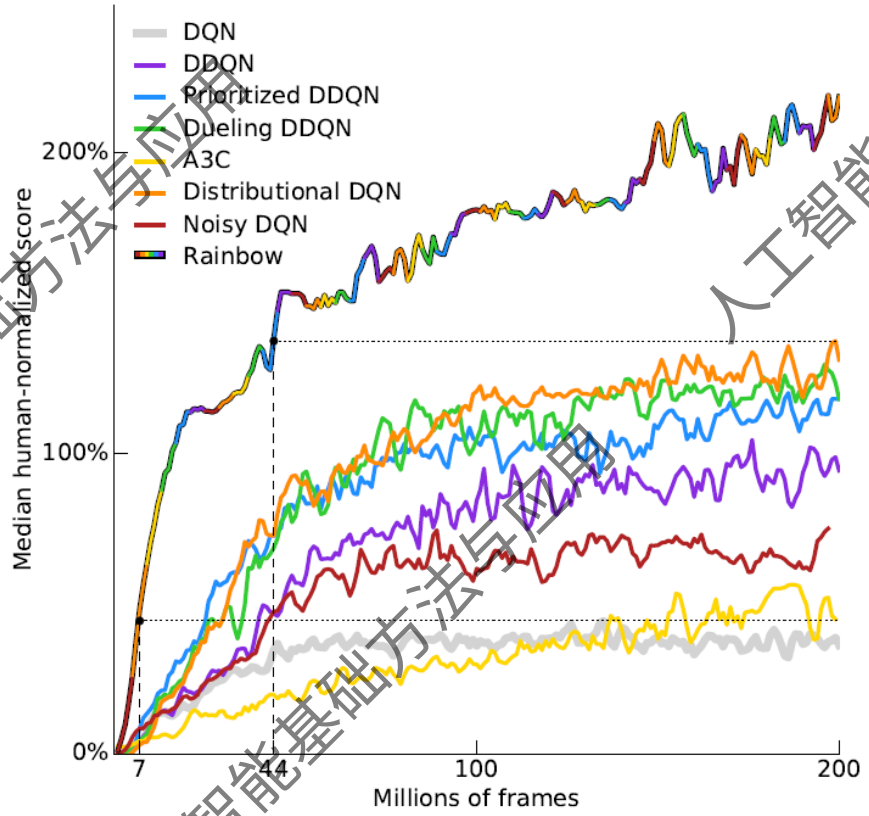
$\pi = \pi'$

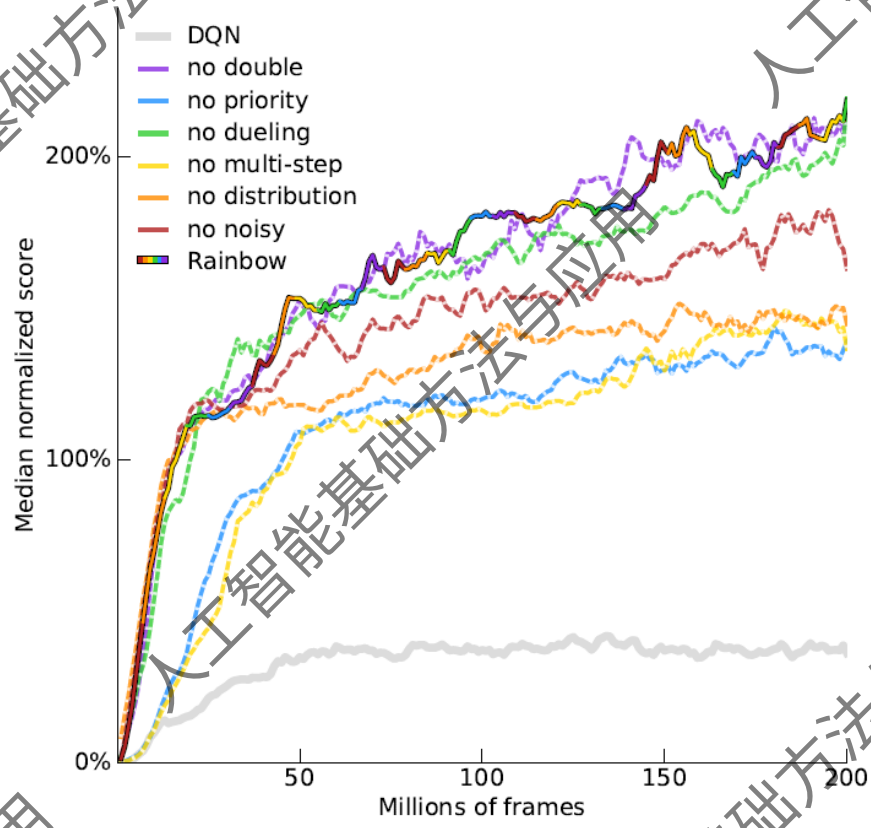
Off-policy

深度Q网络流程

- 初始化函数Q，目标函数$\hat{Q}$，令$\hat{Q} = Q$
- 对于每一个回合
  - 对于每一个时间步t
    - 给定状态s_t，根据Q函数采用贪心策略选择动作a_t
    - 获得奖励r_t，并转移到新状态s_{t+1}
    - 将(s_t, a_t, r_t, s_{t+1})存入回放缓冲区
    - 从回放缓冲区中采样(s_i, a_i, r_i, s_{i+1}) (缓冲区数据≥batch后，采样小批量数据)
    - 构造目标值：$y = r_i + \max_a \hat{Q}(s_{i+1}, a)$
    - 更新Q函数参数，使$Q(s_i, a_i)$尽可能接近目标值 y（回归问题）
    - 每隔 C步，将目标网络更新为当前 Q 网络$\hat{Q} = Q$

Rainbow

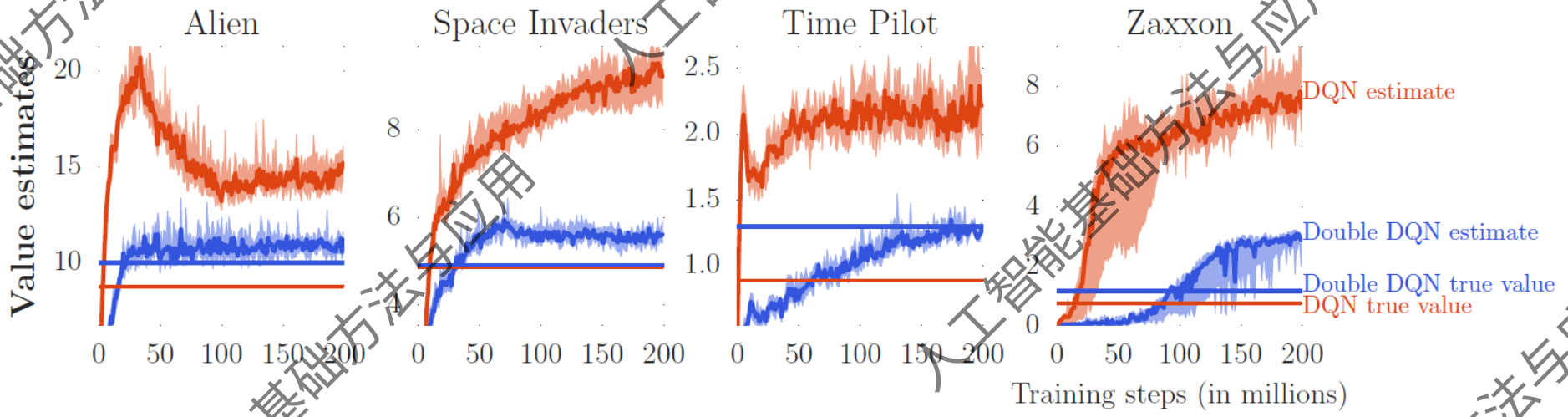




人工智能基础方法与应用

改进1-双深度Q网络 (Double DQN，DDQN)

- Q 值通常会被高估



人工智能基础方法与应用

改进1-双深度DQN

- Q 值通常会被高估

$$Q(s_t, a_t) \longleftarrow r_t + \max_a Q(s_{t+1}, a)$$

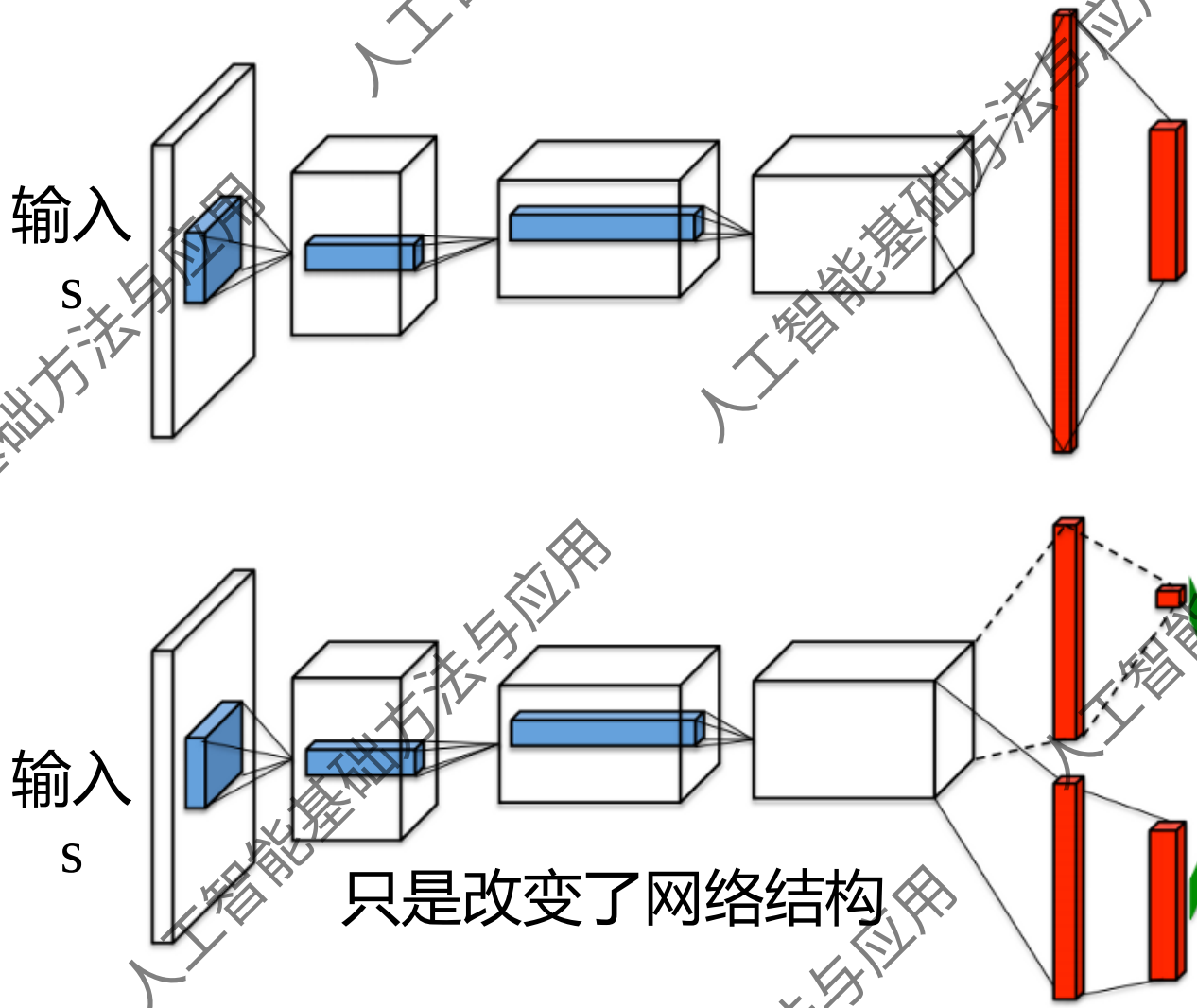
- Double DQN:使用两个函数Q and Q’ 目标网络

$$Q(s_t, a_t) \longleftarrow r_t + Q'\left(s_{t+1}, arg\max_a Q(s_{t+1}, a)\right)$$

如果 Q高估了动作a，该动作可能会被选中；但Q '会对这个动作给出相对独立的估值，从而减轻高估问题.

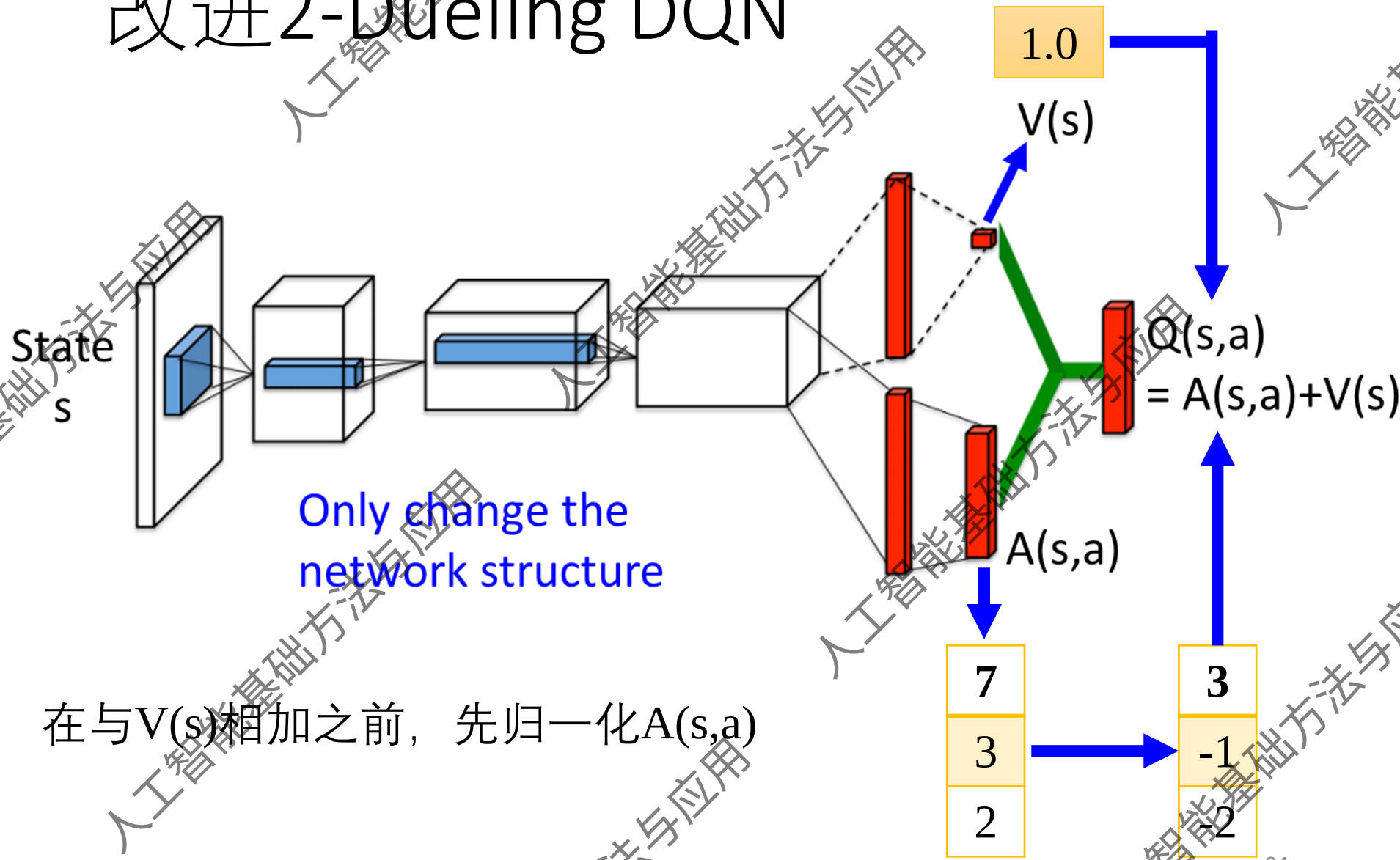
如果 Q'高估了动作，但该动作没有被 Q选中，那么这种高估就不会直接进入目标值。

改进2-竞争深度Q网络 Dueling DQN



人工智能基础方法与应用

改进2-Dueling DQN

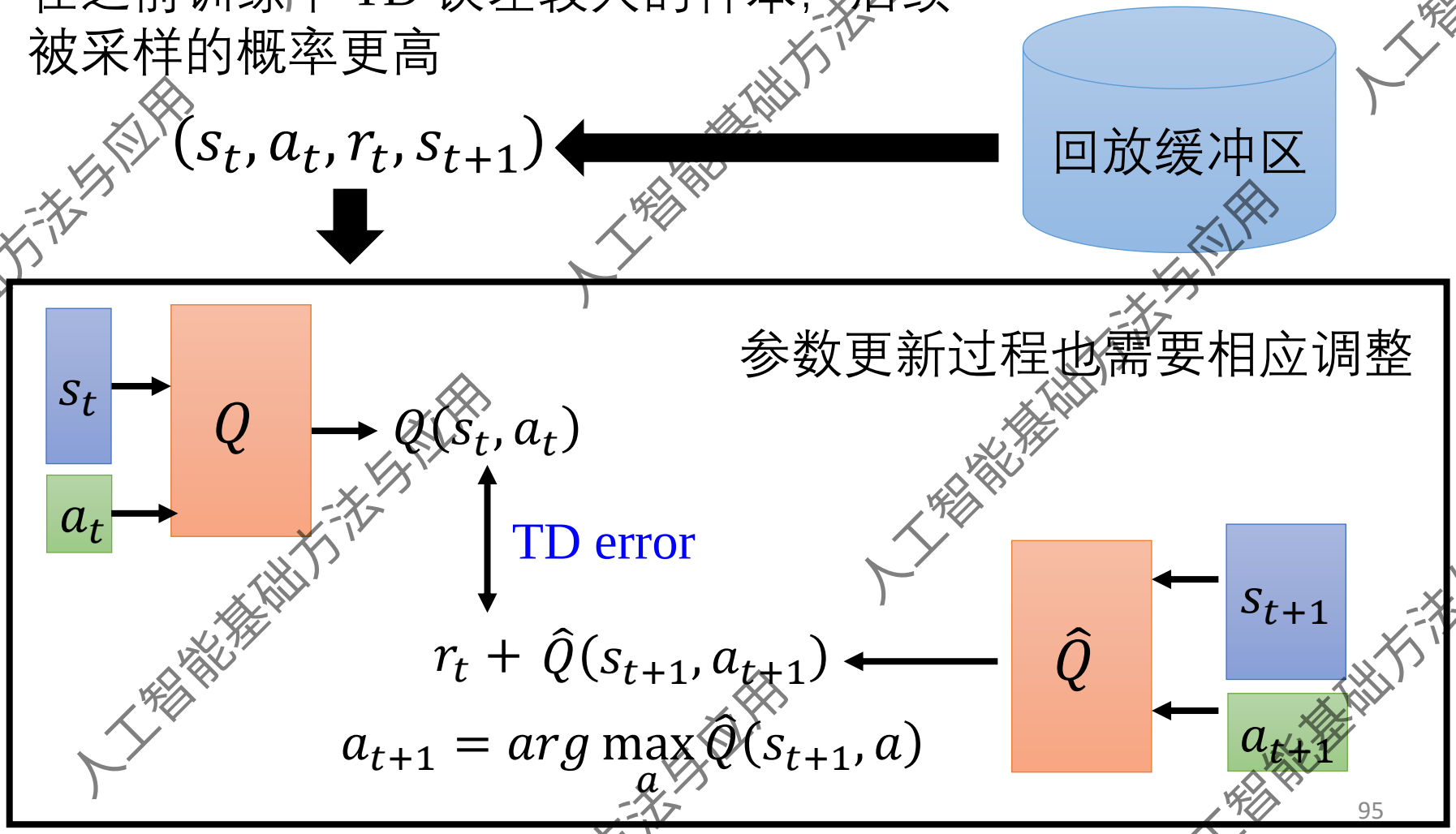


在与V(s)相加之前，先归一化A(s,a)

人工智能基础方法与应用

改进3-优先级经验回放

在之前训练中 TD 误差较大的样本，后续被采样的概率更高



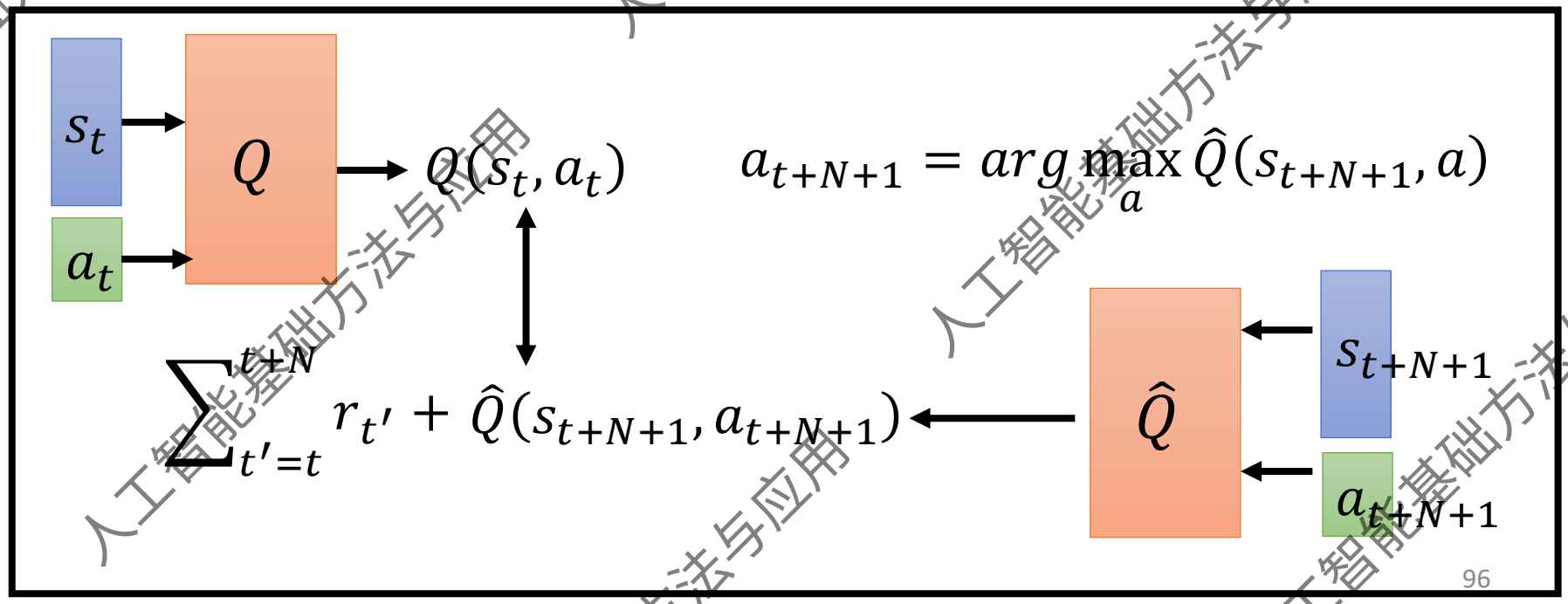
改进4-多步更新

MC和TD之间取得平衡

$(s_t, a_t, r_t, \cdots, s_{t+N}, a_{t+N}, r_{t+N}, s_{t+N+1})$

~~(s_t, a_t, r_t, s_{t+1})~~

回放缓冲区



改进5-噪声网络

- 动作层面的噪声(ε-贪心)

$$a = \begin{cases} arg\max_a Q(s,a), & with\ probability\ 1-\varepsilon \\ random, & otherwise \end{cases}$$

- 参数层面的噪声

在每个回合开始时，
将噪声注入Q函数的参数中

$$a = arg\max_a \tilde{Q}(s,a)$$

$$Q(s,a) \xrightarrow{\text{增加噪声}} \tilde{Q}(s,a)$$

在同一个回合中，噪声**不会**发生变化

改进5-噪声网络

- 动作层面的噪声
  - 面对相同状态，智能体可能采取不同动作
    - 真实策略通常不会以这种完全随机的方式行动
- 参数层面的噪声
  - 面对相同或相似状态，智能体倾向于采取一致动作
    - →s与状态相关的探索
  - 以更一致的方式进行探索

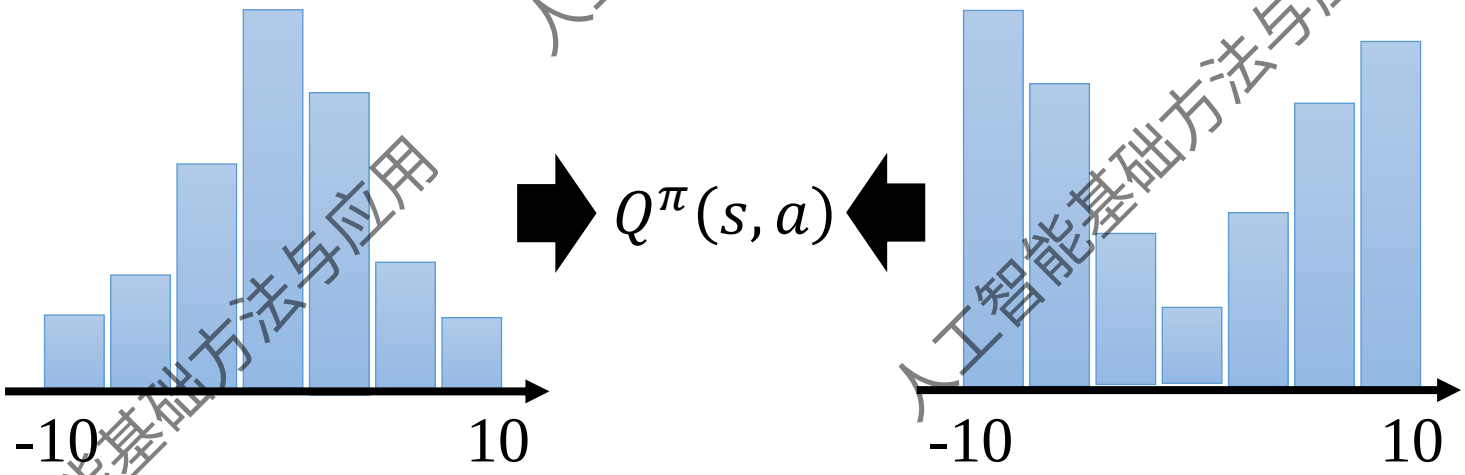
随机乱试

系统地尝试

人工智能基础方法与应用

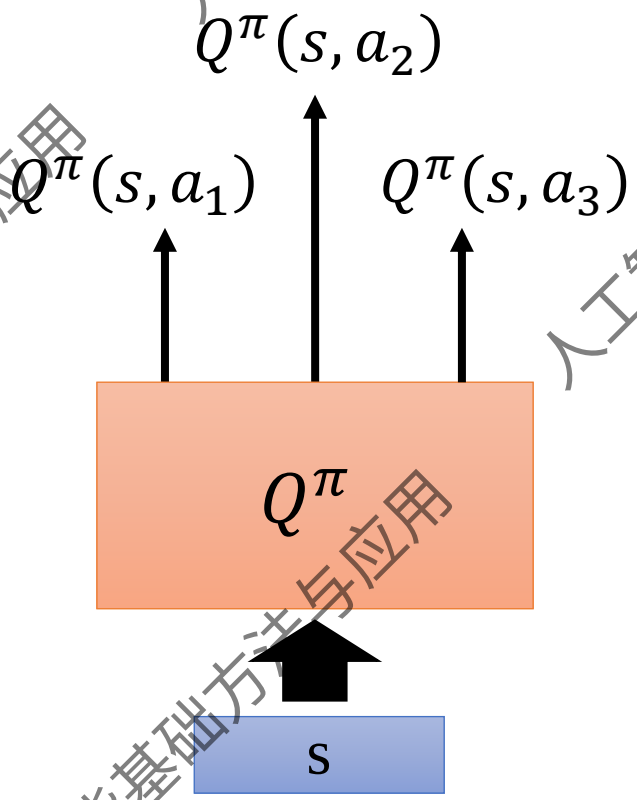
改进6-分布式Q函数

- 状态-动作价值函数$Q^{\pi}(s,a)$
  - 在使用智能体策略π时,智能体看到状态 s 并采取动作a后，未来预期能够获得的累计奖励

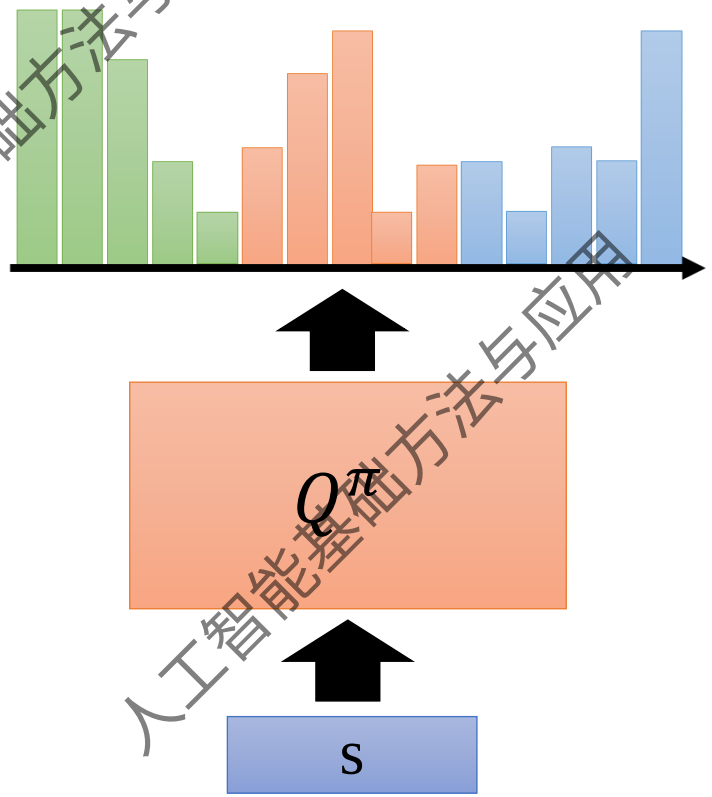


不同的回报分布可能具有相同的期望值

改进6-分布式Q函数



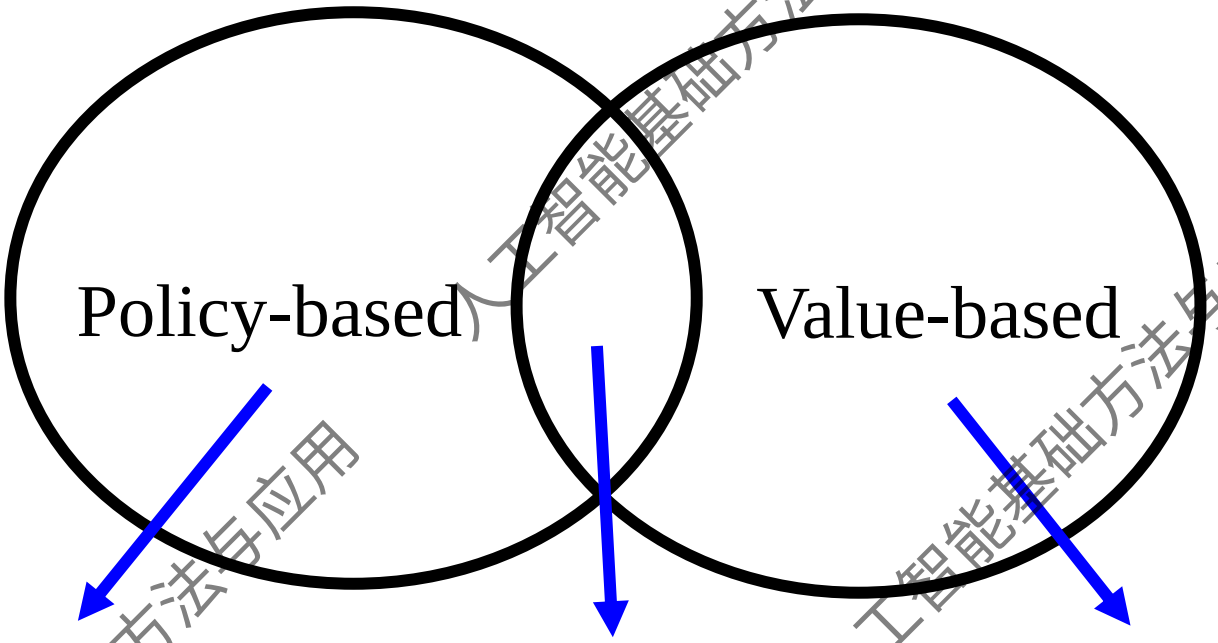
一个具有 3 个输出的网络



一个具有 15 个输出的网络
(每个动作对应 5 个区间)

人工智能基础方法与应用

强化学习方法分类



学习智能体策略
（演员）

演员+评论员
策略网络负责决策，
价值网络负责评价

学习价值函数
（评论员）

人工智能基础方法与应用

目录

RL是什么？（ML的三步骤）

策略梯度

演员-评论员

稀疏奖励

无奖励：从演示中学习

人工智能基础方法与应用

稀疏奖励

$$\mathrm{A}_t = r_t + \gamma V^{\theta}(s_{t+1}) - V^{\theta}(s_t)$$

s_1	s_2	s_3		s_N
a_1	a_2	a_3	$\cdots$	a_N
r_1	r_2	r_3		r_N

Training Data

$\{s_1, a_1\}$	A_1
$\{s_2, a_2\}$	A_2
$\{s_3, a_3\}$	A_3
$\vdots$	$\vdots$
$\{\mathrm{s_N}, \mathrm{a_N}\}$	A_N

如果大多数情况下$r_t = 0$ ➡ 在奖励稀疏的情况下，很难判断每个动作是否真正有价值

例如，让机械臂完成拧螺丝任务

开发者额外设计一些奖励，用来引导智能体学习 ➡ ***设计奖励***

稀疏奖励



选择“玩耍”，
$r_{t+1} = 1, r_{t+100} = -100$

选择“学习”，
~~$r_{t+1} = 1$~~, $r_{t+100} = 100$

$r_{t+1} = 1$

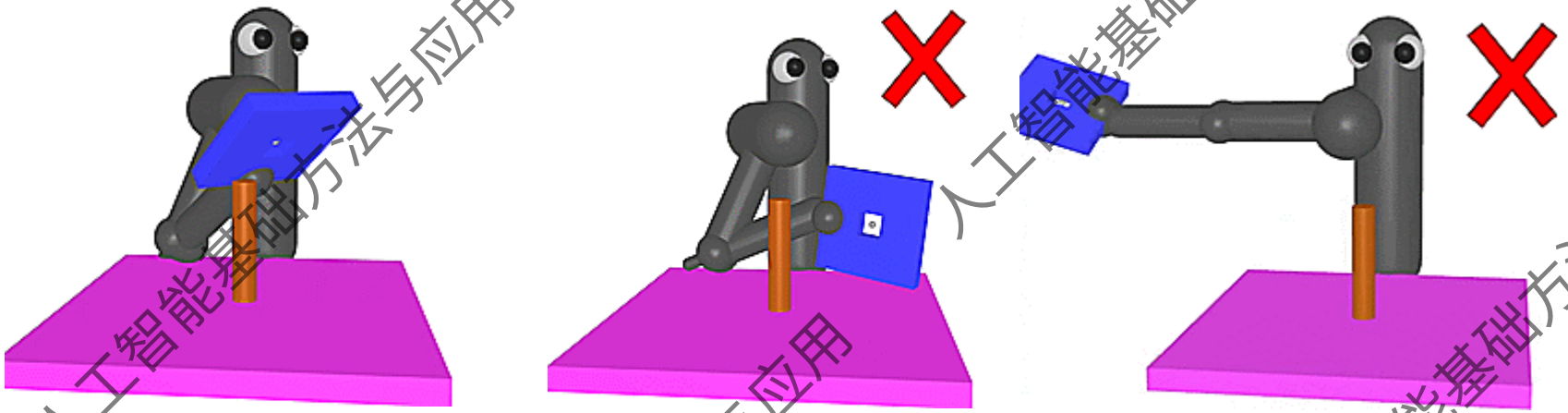


人工智能基础方法与应用

稀疏奖励

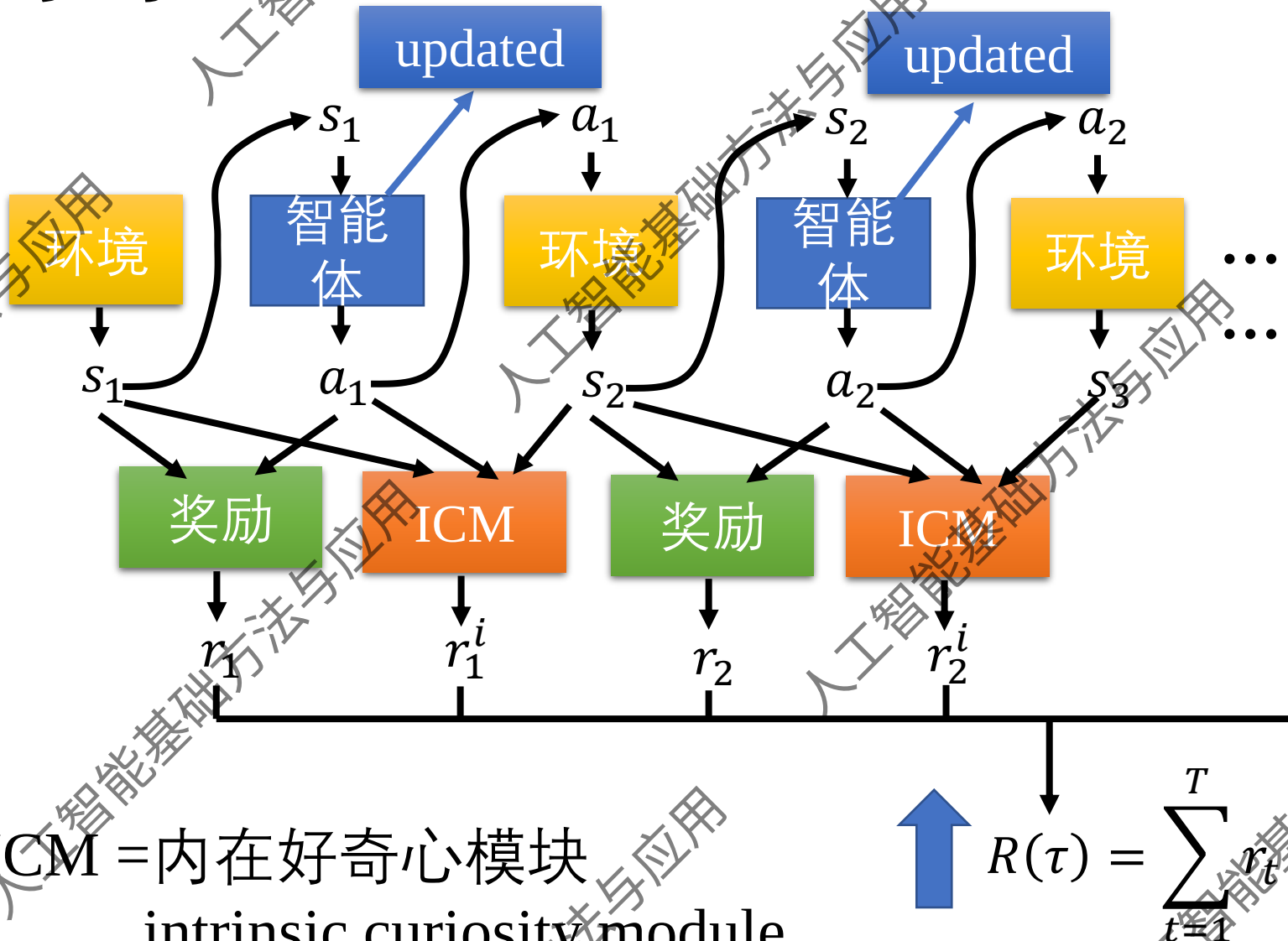
VizDoom

Parameters	Description	FlatMap | CIGTrack1
living	Penalize agent who just lives	-0.008 / action
healthloss	Penalize health decrement	-0.05 / unit
ammo_loss	Penalize ammunition decrement	-0.04 / unit
health_pickup	Reward for medkit pickup	0.04 / unit
ammo_pickup	Reward for ammunition pickup	0.15 / unit
dist_penalty	Penalize the agent when it stays	-0.03 / action
dist_reward	Reward the agent when it moves	9e-5 / unit distance



https://bair.berkeley.edu/blog/2017/12/20/reverse-curriculum/

好奇心



ICM =内在好奇心模块

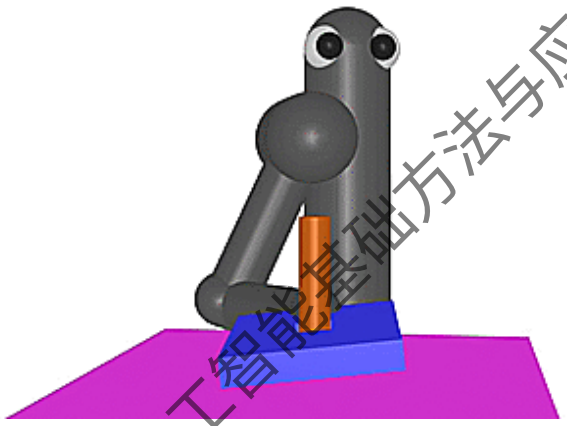
intrinsic curiosity module

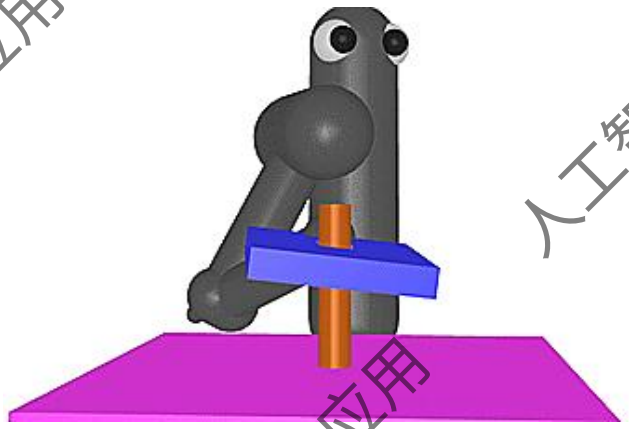
$$R(\tau) = \sum_{t=1}^{T} r_t + r_t^i$$

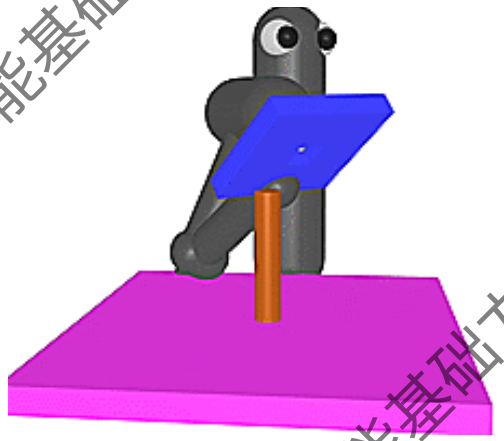
课程学习

- 从简单训练样本开始，然后逐步增加任务难度

	Class 0	Class 1	Class 2	Class 3	Class 4	Class 5	Class 6	Class 7
Speed	0.2	0.2	0.4	0.4	0.6	0.8	0.8	1.0
Health	40	40	40	60	60	60	80	100





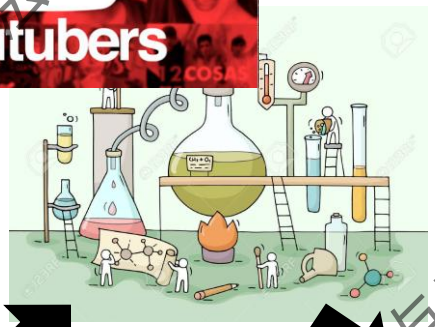


人工智能基础方法与应用

分层强化学习







校长

教授

学生

提供目标

提供子目标

动作

- 如果低层智能体无法完成目标，高层智能体会受到惩罚
- 如果智能体到达了错误目标，则可以认为原先设定的目标存在问题

目录

RL是什么？（ML的三步骤）
策略梯度
演员-评论员
设计奖励
无奖励：从演示中学习

人工智能基础方法与应用

动机

- 在一些复杂任务中，如何定义奖励本身就是一个难题
- 人工设计的奖励可能导致不可控行为



机器人三定律:

1、机器人不得伤害人类，也不得因不作为而使人类受到伤害

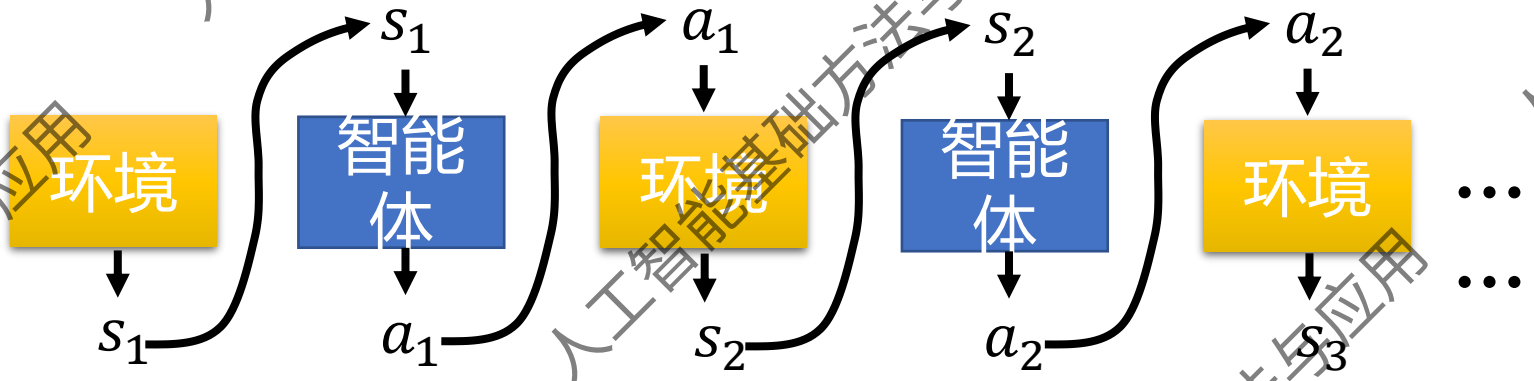
2、机器人必须服从人类命令，除非这些命令与第一定律相冲突

3、在不违背第一定律和第二定律的前提下，机器人应保护自身存在

?????

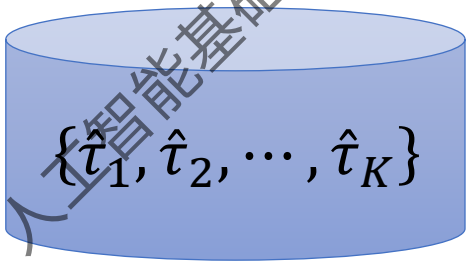
如果只从“保护人类整体生存”目标出发，智能体可能推导出限制个体自由、甚至牺牲少数人的极端行为

模仿学习



智能体可以与环境交互，但奖励函数不可用

我们拥有专家示范数据



每个$\hat{\tau}$都是一条专家轨迹

自动驾驶：记录人类驾驶员的驾驶行为

机器人：可以通过牵引机械臂的方式提供示范

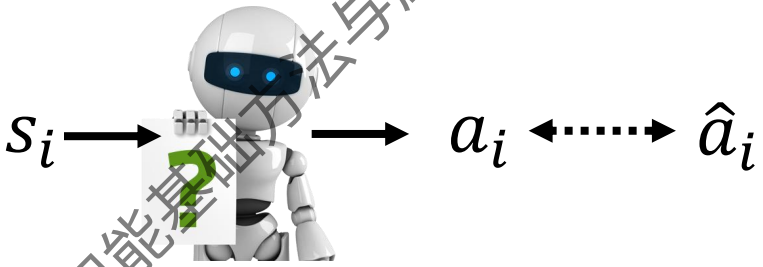
这不就是监督学习吗？

- 以自动驾驶汽车为例

$$\hat{\tau} = \{s_1, \hat{a}_1, s_2, \hat{a}_2, \cdots\}$$

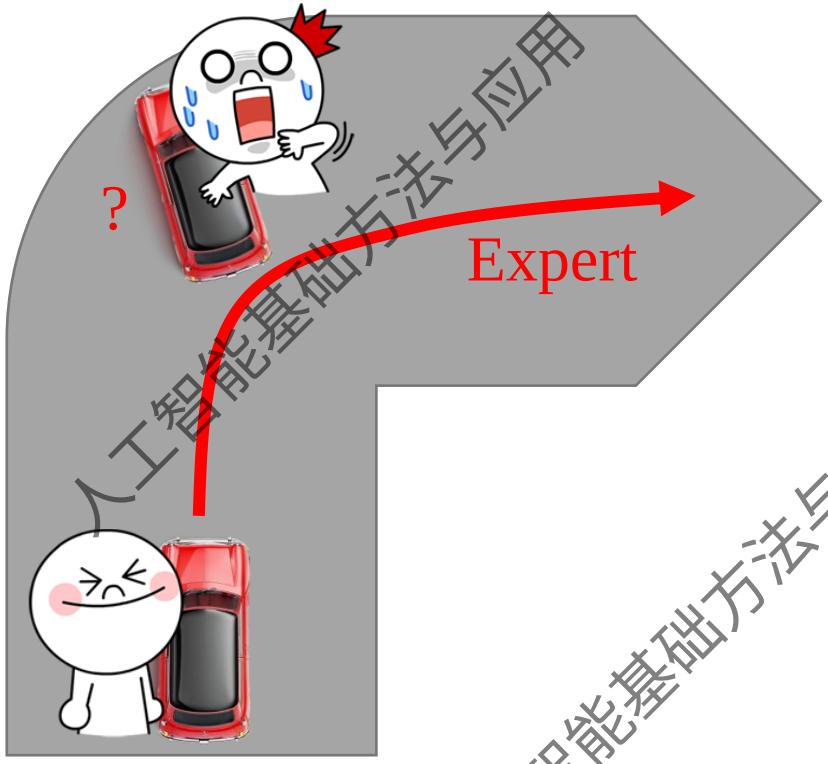


向前



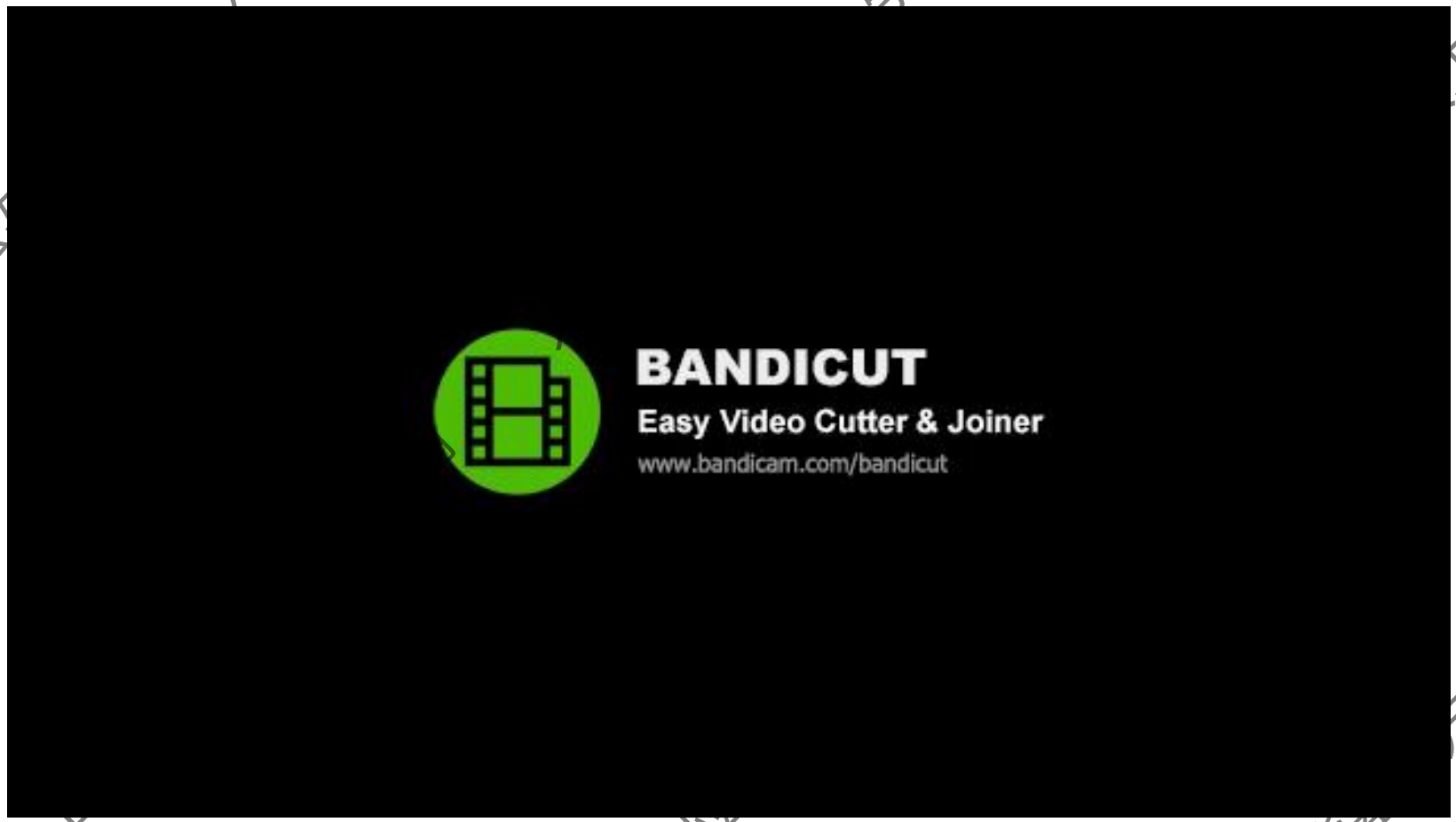
问题：专家数据只能覆盖有限场景，智能体一旦遇到未见过的状态，就可能无法正确应对

是的，这种方法也称为行为克隆



更多问题……

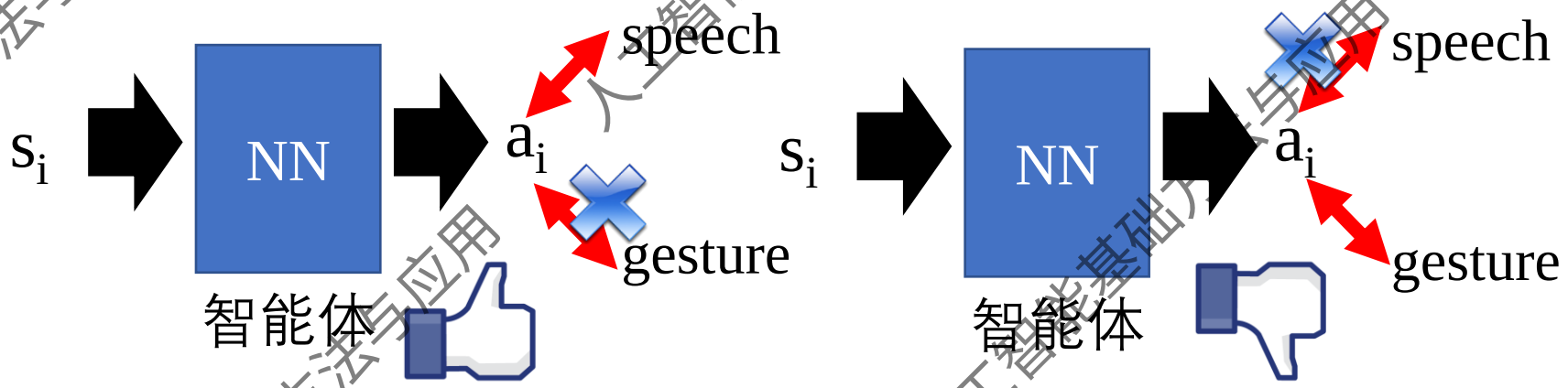
智能体可能会照搬专家的所有动作，连无关动作也一并学进去



https://www.youtube.com/watch?v=j2FSB3bseek

行为克隆

- 主要问题是：如果模型能力有限，它可能会学到错误的行为



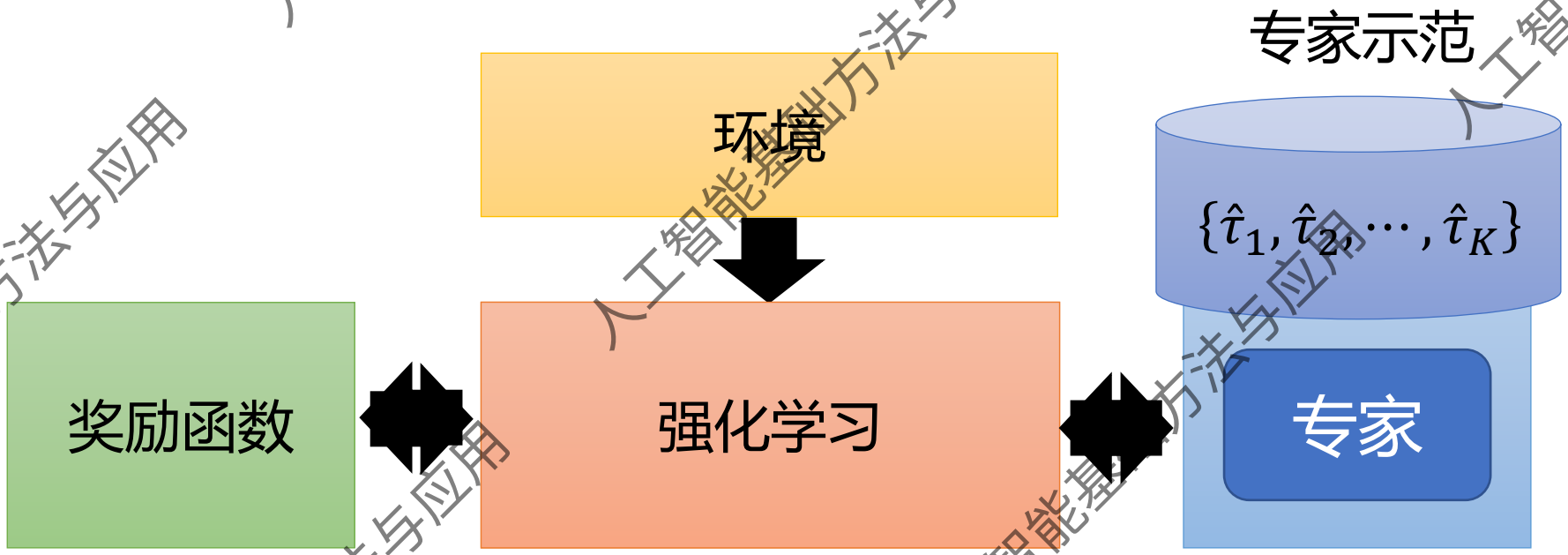
- 有些行为必须模仿，但有些行为其实可以忽略
  - 监督学习会把所有错误同等对待

分布不匹配



- 在监督学习中，我们通常假设训练数据和测试数据来自相同分布
- 在行为克隆中:
  - 训练阶段: $(s, a) \sim \hat{\pi}$ (专家)
    - ***智能体采取的动作(a)会影响后续状态(s)的分布***
  - 测试阶段: $(s', a') \sim \pi^*$ (模仿专家的智能体策略)
    - 如果$\hat{\pi} = \pi^*$, (s, a) 和 (s', a') 来自相同分布
    - 如果$\hat{\pi}$ 和π^* 存在差异，那么s和s'可能相差非常大

逆强化学习



利用奖励函数寻找***最优智能体***.

人工智能基础方法与应用

逆强化学习

- 基本假设：***专家示范优于当前智能体行为***
- 基本思想：
  - 初始化演员
  - 在每次迭代中
    - 智能体与环境交互以获得轨迹
    - 定义奖励函数，使得专家轨迹获得的奖励高于智能体当前生成的轨迹
    - 智能体基于新的奖励函数继续学习，使获得的奖励最大化
    - 最终输出学习到的奖励函数，以及由该奖励函数训练得到的智能体策略

人工智能基础方法与应用

停车场导航



- 奖励函数:
  - 前进驾驶与倒车驾驶
  - 前进与倒车之间的切换次数
  - 车道保持On-road vs. off-road
  - 路径曲率

人工智能基础方法与应用



人工智能基础方法与应用

课堂总结—强化学习
在交互中学习，在反馈中强化

1. 从“直接试错”到“有人评价”

- **Actor**：负责选择动作
- **Critic**：负责评价状态或动作好坏
- **优势函数**：判断某个动作是否优于平均水平

2. 从“完整回合”到“边走边学”

- **MC 方法**：等回合结束，用真实累计奖励学习
- **TD 方法**：每走一步，用当前奖励和下一状态价值更新
- **核心权衡**：MC 方差大，TD 效率高但依赖估计

3. 从“评价动作”到“改进策略”

- **Q 函数**：评价在状态下采取某个动作的长期价值
- **策略改进**：选择 Q 值最大的动作，使策略不断变好
- **DQN 技巧**：目标网络、经验回放、探索机制

4. 从“有奖励”到“奖励不足”

- **稀疏奖励**：奖励塑形、好奇心、课程学习、分层强化学习
- **无显式奖励**：行为克隆、数据集聚合、逆强化学习

课堂总结—强化学习

在交互中学习，在反馈中强化

- **问题特点**：没有明确标准答案，只能通过环境奖励评价行为好坏
- **基本框架**：观测 → 动作 → 奖励 → 新观测，连续交互形成轨迹
- **优化目标**：最大化整个回合的长期累计奖励，而不是单步即时收益
- **策略梯度**：用奖励或优势值判断动作好坏，提高好动作概率，降低差动作概率
- **关键机制**：累计奖励、折扣因子、baseline、探索、on-policy / off-policy

人工智能基础方法与应用